

Master's Programme in Security and Cloud Computing

Reachability Modeling for Kubernetes Network Policies

Metehan Koç

© 2026

This work is licensed under a [Creative Commons](#)
“Attribution-NonCommercial-ShareAlike 4.0 International” license.



Author Metehan Koç

Title Reachability Modeling for Kubernetes Network Policies

Degree programme Security and Cloud Computing

Major Security and Cloud Computing

Thesis supervisor Professor Tuomas Aura (Aalto University), Professor Davide Balzarotti (EURECOM)

Thesis advisor Jose Luis Martin Navarro

Date 30 July 2026

Number of pages 76

Language English

Abstract

As Kubernetes becomes the standard platform for running containerized microservices, network policies are a central mechanism for restricting communication between workloads. In practice, however, it is difficult for operators to determine what connectivity a given policy set actually permits, particularly when policies interact across ingress and egress directions, are expressed in different CNI formats, or contain redundant rules. Existing network verification tools were developed for traditional IP networks and do not support offline analysis of Kubernetes application and policy manifests. This thesis presents an offline control-plane verification framework that analyses application and network policy YAML files without requiring a running cluster or live traffic. The tool normalizes native Kubernetes `NetworkPolicy` resources and Cilium policy CRDs into a common representation, constructs a reachability matrix of allowed and denied communication, and supports connectivity queries and semantic policy comparison. The framework is implemented in Python and evaluated on two reference microservice applications using manually annotated test oracles. The results show that offline control-plane verification offers a practical way to validate network security configurations before deployment.

Keywords Kubernetes, network policies, network verification, reachability matrix, connectivity, Cilium, cloud, security

Auteur	Metehan Koç	
Titre	Reachability Modeling for Kubernetes Network Policies	
Programme d'études	Security and Cloud Computing	
Spécialité	Security and Cloud Computing	
Superviseurs	Professor Tuomas Aura (Aalto University), Professor Davide Balzarotti (EURECOM)	
Conseiller	Jose Luis Martin Navarro	
Date	19 juillet 2026	Nombre de pages 76
		Langue Anglais

Résumé

Alors que Kubernetes s'impose comme la plateforme de référence pour l'exécution de microservices conteneurisés, les politiques réseau constituent un mécanisme central pour restreindre la communication entre charges de travail. En pratique, il reste toutefois difficile pour les opérateurs de déterminer quelle connectivité un ensemble de politiques autorise réellement, notamment lorsque les politiques interagissent entre les directions d'entrée (ingress) et de sortie (egress), sont exprimées dans différents formats CNI, ou contiennent des règles redondantes. Les outils existants de vérification réseau ont été conçus pour les réseaux IP traditionnels et ne permettent pas d'analyser hors ligne les manifests d'applications et de politiques Kubernetes. Cette thèse présente un cadre de vérification hors ligne du plan de contrôle qui analyse des fichiers YAML d'applications et de politiques réseau sans nécessiter de cluster en cours d'exécution ni de trafic réel. L'outil normalise les ressources natives Kubernetes NetworkPolicy et les CRD de politiques Cilium dans une représentation commune, construit une matrice d'atteignabilité des communications autorisées et refusées, et prend en charge des requêtes de connectivité ainsi qu'une comparaison sémantique des politiques. Le cadre est implémenté en Python et évalué sur deux applications de microservices de référence à l'aide d'oracles de test annotés manuellement. Les résultats montrent que la vérification hors ligne du plan de contrôle offre un moyen pratique de valider les configurations de sécurité réseau avant leur déploiement.

Mots-clés	Kubernetes, politiques réseau, vérification réseau, matrice d'atteignabilité, connectivité, Cilium, cloud, sécurité
------------------	---

Preface

This thesis marks the end of my studies in the Security and Cloud Computing (SECCLO) Master's programme. Completing it would not have been possible without the support of many people, and I am grateful for the chance to thank them here.

I would like to thank Professor Tuomas Aura for his supervision and guidance throughout this work. I am especially grateful to my advisor, Jose Luis Martin Navarro, for his continuous support, careful feedback, and patience. His help shaped both the technical direction of the thesis and my understanding of how to turn ideas into a coherent piece of research.

I also thank my friends from the SECCLO programme. Studying and working alongside them made this journey much more enjoyable, and their companionship through courses, projects, and shared challenges meant a great deal to me.

Special thanks go to high school my friends from "Oyun Sevenler Derneği". Playing computer games together, sharing laughs, and simply having fun gave me the balance and energy I needed during the more demanding periods of this thesis. Those moments away from work helped me stay motivated and keep perspective.

Finally, I thank my family for their constant support, encouragement, and belief in me. Without them, this work would not have been completed.

Espoo, 26 July 2026

Metehan Koç



**Co-funded by
the European Union**

Contents

Abstract	3
Preface	5
Contents	6
List of acronyms and abbreviations	8
1 Introduction	9
2 Cloud Computing	11
2.1 Containers	11
2.2 Kubernetes	12
2.3 Kubernetes Network Model and Kubernetes Network Policies	15
2.4 CNI	17
2.4.1 CNI Plugins	17
2.4.2 Comparison	21
3 Network Verification Approaches	23
3.1 Control Plane Approaches	23
3.2 Data Plane Approaches	26
3.3 Comparison	29
4 Methodology	31
4.1 Research Approach and Proposed Solution	31
4.2 Evaluation Design	32
5 Design and Implementation	35
5.1 Workload and Policy Parser	36
5.2 Reachability Matrix Creation	42
5.3 Connectivity Query	49
5.4 Policy Simplification	53
6 Evaluation	56
6.1 Correctness	56
6.1.1 Test Design and Validation Approach	56
6.1.2 Reachability Matrix Validation	58
6.1.3 Connectivity Query Validation	60
6.1.4 Policy Simplification Validation	60
6.2 Performance	61
6.2.1 Computational Complexity	62
6.2.2 Performance Benchmark Setup	62
6.2.3 Performance Results	63

7	Discussion	67
7.1	Practical Implications	67
7.2	Challenges	68
7.3	Limitations	69
7.4	Future Work	69
8	Conclusions	71

List of acronyms and abbreviations

ACL	Access Control List
AD	Administrative Distance
API	Application Programming Interface
ARC	Abstract Representation for Control Planes
BDD	Binary Decision Diagram
BGP	Border Gateway Protocol
CNCF	Cloud Native Computing Foundation
CNI	Container Network Interface
CPU	Central Processing Unit
CRD	Custom Resource Definition
CSV	Comma-separated Values
DNS	Domain Name System
DSR	Direct Server Return
eBPF	extended Berkeley Packet Filter
EC	Equivalence Class
ECMP	Equal-Cost Multi-Path
ETG	Extended Topology Graph
FIB	Forwarding Information Database
HSA	Header Space Analysis
IP	Internet Protocol
IPVS	IP Virtual Server
IR	Intermediate Representation
JSON	JavaScript Object Notation
LLVM	Low Level Virtual Machine
MAC	Medium Access Control
OS	Operating System
OSI	Open Systems Interconnection
OSPF	Open Shortest Path First
OVS	Open vSwitch
RAM	Random Access Memory
RFC	Request for Comments
RIB	Routing Information Base
SAT	Boolean Satisfiability
SDN	Software-defined Networking
SMT	Satisfiability Modulo Theories
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
VM	Virtual Machine
VXLAN	Virtual Extensible Local Area Network
YAML	YAML Ain't Markup Language

1 Introduction

Kubernetes has become the dominant platform for deploying and operating containerized applications in production. In the 2025 Cloud Native Computing Foundation (CNCF) annual survey, 82% of container users reported using Kubernetes in production, up from 66% in 2023 [8, 9]. As organizations adopt microservice architectures, applications are split into many workloads that run as pods across namespaces and communicate over the cluster network. Securing such deployments involves several concerns; one common requirement is network segmentation, that is, restricting which components are allowed to exchange traffic. Network policies are a central mechanism for this form of control: they restrict pod-to-pod traffic using declarative rules based on labels, namespaces, ports, protocols, and Internet Protocol (IP) ranges [27]. By limiting which components may communicate, network policies reduce the attack surface of a cluster and help prevent lateral movement between services [5]. In practice, however, securing a microservice deployment is not only a matter of writing policies but of understanding the resulting *connectivity*: which workloads may send traffic to which other workloads, under which ports and protocols, once all applicable rules are taken into account.

Determining this effective connectivity in a real Kubernetes environment is difficult. By default, Kubernetes permits unrestricted communication across the pod network unless additional controls are applied [44]. Once policies are introduced, the resulting behaviour depends on how multiple ingress and egress rules interact across workloads, namespaces, and traffic directions. Policies may be incomplete, inconsistent, or redundant relative to one another [33, 34], and clusters may contain heterogeneous policy formats when Container Network Interface (CNI) plugins extend the native Kubernetes model with explicit deny rules, cluster-wide scope, and additional Custom Resource Definition (CRD)-based policy types [7, 22]. Operators therefore need systematic ways to verify which communication complies with a given policy set before application or policy updates are applied to a running cluster. Prior research on network verification offers two broad approaches, but these were developed primarily for traditional IP networks built from physical and virtual switches, routers, firewalls, and routing protocols such as Border Gateway Protocol (BGP) and Open Shortest Path First (OSPF) [2, 15, 19, 21]. Their models assume packet forwarding through network devices and vendor-specific configuration files, rather than the label-selected pods, namespace-scoped YAML manifests, and CNI-specific policy semantics of a Kubernetes cluster. *Control-plane* methods analyze configuration and routing logic to infer forwarding behaviour proactively [2, 12, 15, 16], but they do not directly capture Kubernetes abstractions such as pod selectors, service endpoints, or explicit Cilium deny rules. *Data-plane* methods, by contrast, analyze installed or live forwarding state to check properties such as reachability and isolation [19, 21, 35, 48]; in Kubernetes, however, applying such an approach would require network policies to be translated into low-level forwarding rules such as `iptables` or extended Berkeley Packet Filter (eBPF) programs and distributed across live nodes before inconsistencies could be detected. Neither family of tools is designed to reason offline over declarative application and policy manifests in a cloud-native setting.

This thesis adopts a control-plane verification approach tailored to Kubernetes network policies. We propose an offline framework that analyses application manifests and network policy YAML files directly, without requiring a running cluster or live traffic. Application manifests describe the cluster topology, including namespaces, workloads, services, and declared ports, while policy manifests define ingress and egress allow and deny rules. From these inputs, the framework constructs a reachability matrix that makes effective connectivity visible across the application topology. Operators can inspect allowed and denied communication between endpoints, submit connectivity queries for specific source–destination pairs, and compare two policy sets to decide whether they produce the same connectivity, or whether one set contains rules that are redundant with respect to the other. To support heterogeneous deployments, the framework normalizes both native Kubernetes `NetworkPolicy` resources and Cilium policy CRDs into a common internal representation, so the same analysis logic applies regardless of the policy syntax. Cilium-specific semantics, including explicit deny rules and cluster-wide policies, are handled through ordered policy application and priority-based query resolution. This design supports proactive validation of network security configurations before deployment and remains independent of the internal data-plane implementation of a specific CNI plugin.

The main contributions of this work are as follows:

1. A reachability-matrix model for Kubernetes and Cilium network policies that represents connectivity at multiple levels of granularity.
2. A connectivity-query mechanism that resolves reachability questions over wildcard endpoints using a priority-based evaluation model, including support for Cilium explicit deny rules.
3. A policy-comparison capability that enables operators to determine whether two policy configurations produce the same connectivity or contain redundant rules, including policies expressed in different Kubernetes and Cilium formats.

To evaluate the correctness of the proposed solution, this work includes an evaluation on two reference applications using manually annotated test oracles. The evaluation demonstrates correctness across representative Kubernetes and Cilium policy scenarios and reports scalability under increasing policy load.

The remainder of this thesis is organized as follows. Section 2 introduces the background on containers, Kubernetes, network policies, and CNI plugins. Section 3 reviews network verification approaches from the literature. Section 4 presents the research methodology, evaluation design, and testbeds. Section 5 describes the design and implementation of the verification tool. Section 6 reports the correctness and performance evaluation results. Section 7 discusses implementation challenges, limitations, and directions for future work. Section 8 concludes the thesis.

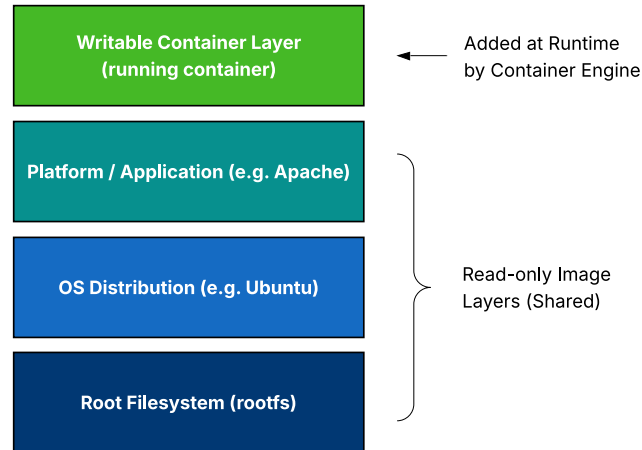


Figure 1: Layered architecture of containers

2 Cloud Computing

This chapter provides an overview of foundational cloud computing concepts and technologies relevant to this work. Section 2.1 introduces containerization principles, followed by Section 2.2, which discusses orchestration using Kubernetes. Section 2.3 explores the Kubernetes network model and network policies. Finally, Section 2.4 examines the CNI, detailing specific CNI plugins and comparing their characteristics.

2.1 Containers

Containers function as a lightweight packaging mechanism that encapsulates an application along with all its necessary dependencies, binaries, and libraries. This design ensures that software runs reliably and consistently across diverse operating systems and computing infrastructures, effectively mitigating the traditional challenge of application instability during environmental migrations [43]. While the concept of containerization originated with the introduction of chroot in 1979, and its first complete implementation emerged in 2007, the technology only achieved widespread popularity with the emergence of Docker in 2013 [37].

Control groups and namespaces are two mechanisms that make containers possible today. Control groups limit the usage of resources of a process, while namespaces prevent processes from accessing each other's resources, working together to isolate containers from other processes on the host. Beyond this process isolation, containers feature a lightweight, layered structure as shown in Figure 1. They are composed of multiple read-only file systems stacked on top of each other using a mechanism called a union mount, which overlays these separate layers so they appear as a single, unified file system. This foundation typically builds upward from a core root file system (rootfs) kernel image, up through an Operating System (OS) distribution layer like Ubuntu, and into specific platform components like Apache. When a container engine launches an image for execution, it adds a single, thin writable layer directly on top of this read-only stack. This top layer represents the executable container itself, allowing

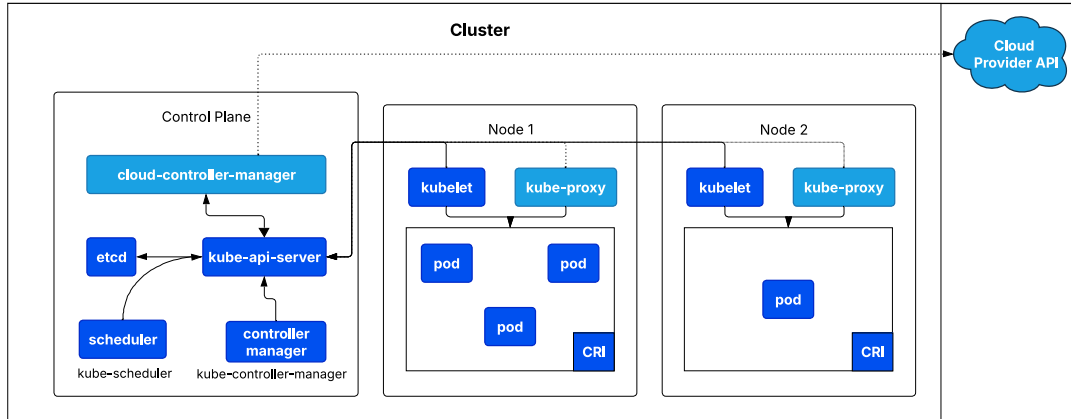


Figure 2: Kubernetes cluster architecture [25]

the application to maintain state and run in its own isolated environment while sharing the static, reusable image layers underneath [38].

Containers are frequently compared with virtual machines (VMs) because both provide virtualization but at different levels. VMs run a complete guest operating system on virtualized hardware, whereas containers share the host operating system and are isolated through OS-level mechanisms. VMs therefore offer hardware-level virtualization, while containers offer OS-level virtualization. Sharing the host OS makes containers more lightweight than VMs [41] and containers have higher performance than the VMs in many cases [13]. This same design choice reduces isolation, however, because containers depend on the host kernel rather than on a separate operating system instance [41]. VMs are often preferred when strong isolation is the primary requirement, whereas containers are typically chosen when efficiency and performance are more important.

2.2 Kubernetes

As an open-source container orchestration system, Kubernetes automates the deployment, scaling, and load distribution of pods, which serve as execution units containing single or multiple containers. It provides core services such as self-healing, service discovery, and secret management, enabling applications to run reliably in distributed environments [30]. Kubernetes is the most widely used container orchestration platform. In the 2025 CNCF annual survey, 82% of container users reported using Kubernetes in production, up from 66% in 2023 [8, 9].

A Kubernetes cluster consists of a control plane and one or more worker nodes, as illustrated in Figure 2. The control plane supervises the cluster, while worker nodes run application workloads as pods. To provide this orchestration, the control plane and worker nodes rely on distinct but interconnected components [25].

Control plane The control plane is responsible for cluster-wide management and decision making. It includes the cloud-controller-manager, etcd, kube-apiserver,

kube-scheduler, and kube-controller-manager [25]:

- **cloud-controller-manager:** Integrates the cluster with an external cloud platform by interacting with that provider's Application Programming Interface (API) and by running controllers that manage cloud-dependent resources, so that cloud-specific logic remains separate from the core control plane.
- **etcd:** Stores cluster data as a distributed, strongly consistent key-value database, including configuration, runtime state, and metadata.
- **kube-apiserver:** Serves as the main entry point to the control plane by exposing the Kubernetes API and processing client requests, typically over REST.
- **kube-scheduler:** Decides where new pods should run by placing them on suitable nodes according to capacity, constraints, and scheduling rules.
- **kube-controller-manager:** Hosts controllers that repeatedly compare the observed cluster state with the desired state declared through the API and act to reduce any difference [26].

Worker node Application workloads run on worker nodes. Each node hosts a kubelet, kube-proxy, and a container runtime [25]:

- **kubelet:** A per-node agent that starts and monitors the containers belonging to pods scheduled on that node.
- **kube-proxy:** Configures local networking so that pods can communicate and so that Kubernetes Services can forward traffic correctly.
- **Container runtime:** Provides the low-level mechanism for creating, running, and stopping containers on the node.

Kubernetes utilizes "objects" to represent the intended state of a cluster. These objects define the nature of containerized applications and the policies governing their behavior. Created, modified, and deleted via the Kubernetes API, objects prompt the system to continuously work to ensure their declared state exists. Users manage these resources either imperatively through the kubectl command-line interface or, more commonly, by defining them declaratively in YAML configuration files as presented in Figure 3 [31]. As shown in Figure 3, a Kubernetes manifest contains four top-level fields, each of which may include nested subfields. The field `apiVersion` specifies the API version used to create the object, `kind` defines the object type, `metadata` contains identifying information such as the namespace and labels, and `spec` describes the desired state of the object.

Kubernetes defines many object types, including workloads, Services, and policies. A workload is an application whose life cycle is managed by Kubernetes. In practice, applications are rarely deployed as individual Pods. Instead, workload resources such as Deployments, ReplicaSets, StatefulSets, DaemonSets, Jobs, and CronJobs are used because their controllers manage Pod creation, health, and desired replica count. Each of these workload resources offers unique features tailored to specific use cases [32]:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25.4
          ports:
            - containerPort: 80
```

Figure 3: Example Kubernetes deployment YAML

- **Deployment and ReplicaSet:** Deployment is used to run stateless applications. When deployment is created, it creates a ReplicaSet to make sure that the desired number of pods exist.
- **StatefulSet:** It is used to run applications with states. It assigns PersistentVolumes as storage to keep the states of the pods.
- **DaemonSet:** It ensures that a specific Pod runs on all (or selected) nodes within a Kubernetes cluster, automatically scaling up or down as nodes are added or removed. It is typically used for background system services that need to run everywhere,
- **Job and CronJob:** A Job executes a specific task to completion exactly once and then stops. A CronJob builds on this by automatically running the same Job multiple times based on a recurring, predefined schedule.

2.3 Kubernetes Network Model and Kubernetes Network Policies

Network Model Kubernetes assigns each pod a unique IP address and, by default, allows pods to communicate with one another without restriction, both on the same node and across nodes [44]. This is commonly described as a flat, permissive pod network: unless additional controls are applied, any pod can reach any other pod. Communication between containers within the same pod is handled separately through localhost and is not affected by network policies [44]. Because pod IP addresses change as pods are recreated, applications usually access workloads through Services rather than directly through pod IPs. A Service provides a stable virtual IP and Domain Name System (DNS) name for a dynamically changing set of pods selected by labels [29]. Kubernetes defines four Service types [29]:

- **ClusterIP:** Makes the Service reachable only from within the cluster and is used unless another type is specified.
- **NodePort:** Publishes the Service through a fixed port that is opened on each node.
- **LoadBalancer:** Creates an outside load balancer and directs incoming traffic to the Service in the cluster.
- **ExternalName:** Associates the Service with an outside DNS name by means of a CNAME record.

Network Policies Network policies provide a mechanism to restrict pod-to-pod connectivity. They operate at Open Systems Interconnection (OSI) layers 3 and 4 and define ingress and egress rules for pods selected by labels. A rule specifies which sources may send traffic to selected pods (ingress) or which destinations selected pods

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-network-policy
  namespace: default
spec:
  podSelector:
    matchLabels:
      role: backend
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: frontend
      ports:
        - protocol: TCP
          port: 8080
  egress:
    - to:
        - podSelector:
            matchLabels:
              role: db
      ports:
        - protocol: TCP
          port: 5432

```

Figure 4: Example Kubernetes network policy restricting ingress and egress for backend pods

may contact (egress); the permitted peers may be defined as pods, namespaces, or external IP ranges [27].

Importantly, network policies do not limit communication between containers inside the same pod. They apply only to traffic that crosses the pod boundary. By default, Kubernetes provides a flat network in which all pods can reach one another, including pods in different namespaces within the same cluster. If no network policy selects a pod, that pod remains fully reachable under this model. Once a pod is selected by a network policy, however, it becomes isolated for the directions declared in `policyTypes`: traffic is allowed only if explicitly permitted by a matching rule. When multiple policies apply to the same pod, the resulting connectivity is the union of all allowed rules.

Figure 4 illustrates an example Kubernetes network policy. The policy is created in a namespace and uses `podSelector` to choose the pods it protects in this case, pods labelled `role: backend`. The `policyTypes` field declares whether the policy contains ingress rules, egress rules, or both. The ingress and egress sections then define the permitted sources and destinations, respectively. In this example, the policy controls which pods labeled `role: backend` may send traffic to other pods and which pods may receive traffic from them.

Kubernetes network policies use three types of selectors to define how traffic flows

to and from pods [27]:

- **podSelector:** It targets specific pods within the same namespace as the NetworkPolicy object.
- **namespaceSelector:** It targets an entire namespace based on its labels. When applied, the policy affects all pods within that selected namespace.
- **ipBlock:** It specifies particular IP Classless Inter-Domain Routing (CIDR) ranges, which is ideal for managing traffic to and from external resources outside the cluster.

2.4 CNI

CNI is a CNCF project that defines a specification and libraries for plugins that configure network interfaces in Linux containers [10]. In Kubernetes, the CNI plugin is responsible for implementing pod networking and, when supported, for enforcing network policies. CNI plugins therefore play an important role in Kubernetes network security. While native Kubernetes network policies operate at OSI layers 3 and 4, some CNI implementations extend this functionality through CRDs, enabling additional capabilities such as layer 7 controls. Some CNIs also support cluster-wide policies that are not restricted to a single namespace, as well as explicit deny rules that are not available in standard Kubernetes network policies[22].

2.4.1 CNI Plugins

Major CNI plugins used in Kubernetes include Cilium [17], Calico [45], Antrea [39], kube-router [24] and Flannel [14]. This thesis focuses on Cilium because it extends Kubernetes network policy enforcement with additional policy types and explicit deny rules.

Cilium is an eBPF-based CNI that provides security, observability, and monitoring capabilities. By running logic in the Linux kernel through eBPF, Cilium can enforce network controls without modifying application code [6]. Network policies are a central part of this security model. Cilium can enforce standard Kubernetes network policies, which are limited to OSI layers 3 and 4, as well as its own policy resources defined as CRDs, which can take effect at layers 3, 4, and 7.

Cilium defines two policy resources: `CiliumNetworkPolicy` and `CiliumClusterwideNetworkPolicy`. As shown in Figure 5, `CiliumNetworkPolicy` has a structure similar to Kubernetes network policies but differs in what it can express. For example, it supports layer 7 rules and explicit deny policies. Deny rules were introduced in Cilium 1.9; in Cilium, an explicit deny takes precedence over an allow rule for the same traffic path. This precedence applies to both `CiliumNetworkPolicy` and `CiliumClusterwideNetworkPolicy`. Standard Kubernetes network policies do not define explicit deny rules in the same way; instead, they rely on implicit denial once a pod is selected by a policy [7]. `CiliumClusterwideNetworkPolicy` is otherwise

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: cilium-pod-selector-egress-ingress
  namespace: backend-ns
spec:
  endpointSelector:
    matchLabels:
      app: makeline-service
  ingress:
    - fromEndpoints:
        - matchLabels:
            app: order-service
  egress:
    - toEndpoints:
        - matchLabels:
            app: product-service
```

Figure 5: Example of a CiliumNetworkPolicy

```

apiVersion: projectcalico.org/v3
kind: NetworkPolicy
metadata:
  name: security-tier.isolate-database
  namespace: production
spec:
  tier: security-tier
  order: 10
  selector: role == 'database'
  types:
  - Ingress
  ingress:
  - action: Allow
    protocol: TCP
    source:
      selector: role == 'frontend'
    destination:
      ports:
      - 5432
  - action: Deny

```

Figure 6: Example of a tiered network policy in Calico

similar to CiliumNetworkPolicy but it is not limited to a single namespace. A cluster-wide policy can affect endpoints across the entire cluster [7].

Calico Calico is as a comprehensive platform providing networking, network security, and observability across any Kubernetes distribution deployed in cloud, on-premises, or edge environments [46]. To achieve this, Calico operates primarily through a node-based agent called Felix, which programs the forwarding information database (FIB) and access control lists (ACLs) to manage interface routing and network policies on each host. These local routes are then communicated across the cluster by the BIRD routing daemon using the Border Gateway Protocol (BGP) for efficient inter-host data distribution, while optimizing components like Typha ensure the platform can scale efficiently to hundreds of nodes.

Calico supports and extends default Kubernetes network policies through its CRDs, offering two distinct policy types: NetworkPolicy and GlobalNetworkPolicy. While NetworkPolicy mirrors the structure of standard Kubernetes network policies, GlobalNetworkPolicy operates cluster-wide across all namespaces, functioning similarly to Cilium’s CiliumClusterwideNetworkPolicy.

Much like Cilium, Calico supports explicit deny actions within its policies. However, a key distinction between the two lies in Calico’s implementation of *tiers*. Tiers allow administrators to define a strict order of precedence for network policies; policies assigned to lower-ordered tiers are evaluated prior to those with higher-ordered designations [47]. An example of a Calico policy incorporating both a deny action and a specific tier configuration is illustrated in Figure 6.

Antrea Antrea is a Kubernetes-native networking and security solution designed to provide pod connectivity and security tracking across diverse cloud and on-premises

environments. To accomplish this, Antrea functions primarily via a node-level daemon called the `antrea-agent`, which acts as the data plane programmer by managing Open vSwitch (OVS) instances running on each host to enforce local interface routing and network policies. These localized node states are coordinated globally by a centralized component called the `antrea-controller`, which watches the Kubernetes API for pods, namespaces, and security rules, computing them into optimized internal network structures to ensure efficient, cluster-wide policy distribution [1].

Antrea supports and extends native Kubernetes network security capabilities through its own Custom Resource Definitions (CRDs), introducing two distinct resources: `AntreaNetworkPolicy` and `ClusterNetworkPolicy`. While `AntreaNetworkPolicy` provides enhanced rules at a localized namespace level, `ClusterNetworkPolicy` operates cluster-wide across all namespaces, offering administrators structural governance similar to Cilium’s cluster-wide configuration constraints [1].

Much like Cilium and Calico, Antrea provides explicit control over traffic constraints by supporting targeted deny and reject actions within its rules. However, unlike Calico, the `antrea-controller` automatically generates five static, read-only system tiers upon startup alongside a baseline resource to dictate structural evaluation order [1].

Kube-router Kube-router is a turnkey networking platform designed to provide an integrated, all-in-one package for pod connectivity, service proxying, and network security across Kubernetes environments. Rather than deploying disjointed components, kube-router minimizes architectural moving parts by bundling these operations into a single lean DaemonSet. The platform utilizes high-performance Linux kernel subsystems, replacing `kube-proxy` with an IP Virtual Server (IPVS) framework for layer-4 load balancing while leveraging the Border Gateway Protocol (BGP) for direct node-to-node routing. This architecture enables advanced capabilities like Direct Server Return (DSR) and Equal-Cost Multi-Path (ECMP) routing to eliminate traffic bottlenecks [23].

In terms of security, kube-router strictly implements the native Kubernetes `NetworkPolicy` specification to safeguard pod communication. Unlike alternatives like Calico or Antrea, it omits proprietary CRDs or multi-layered security tiers. Instead, it reads standard Kubernetes policy declarations directly and translates them into host-level firewall configurations via `iptables`, ensuring a lightweight operational footprint [23].

Flannel Flannel is a simple, lightweight layer-3 network fabric designed specifically to provide pod connectivity across multiple nodes within a Kubernetes cluster. To achieve this, the platform runs a single binary agent called `flannel` on each host, which interacts with the Kubernetes API to allocate a unique subnet lease out of a larger, pre-configured cluster address space. Unlike complex alternatives, Flannel does not control host-to-container networking locally; instead, it acts strictly as a transit fabric that captures cross-host data packets and forwards them using low-level kernel backend mechanisms such as Virtual Extensible Local Area Network (VXLAN)

encapsulation or direct host routing (`host-gw`) [14].

In terms of security enforcement, Flannel is built with a minimal, focused approach that explicitly decouples routing from policy administration. Unlike feature-rich platforms like Calico or Antrea, Flannel completely omits CRDs, security tiers, and built-in `NetworkPolicy` engine implementations. Instead, it relies on external security controllers such as Calico’s policy-only engine to manage access control lists, maintaining a clean architectural separation where Flannel handles underlying traffic transport while third-party integrations apply firewall constraints.

2.4.2 Comparison

CNI	Policy Types	Scope	Layers	Actions	Priority
Flannel	No	No	No	No	No
Kube-router	Kubernetes Policy	Namespace	L3, L4	Allow, Deny	No
Calico	Kubernetes Policy, Calico Policy	Namespace, Clusterwide	L3, L4, L7	Allow, Deny, Log, Pass	Order
Cilium	Kubernetes Policy, Cilium Policy	Namespace, Clusterwide	L3, L4, L7	Allow, Deny	No
Antrea	Kubernetes Policy, Antrea Policy	Namespace, Clusterwide	L3, L4, L7	Allow, Drop, Reject, Pass	Tier

Table 2: Comparison of CNIs policy capabilities

Table 2 compares the policy capabilities of widely used Kubernetes CNI plugins. The comparison is organized along five dimensions: supported policy types, scope, OSI layers, permitted rule actions, and whether policy precedence is explicitly defined.

Policy types and scope Flannel provides basic pod networking only and does not implement network policy enforcement. Kube-router supports standard Kubernetes `NetworkPolicy` resources within a namespace. Calico, Cilium, and Antrea support both native Kubernetes policies and CNI-specific policy resources defined as CRDs. Among these, Calico, Cilium, and Antrea also support cluster-wide policies that are not restricted to a single namespace.

Layers and actions Native Kubernetes network policies operate at OSI layers 3 and 4. Calico, Cilium, and Antrea extend enforcement to layer 7 through their own policy models. The supported actions also differ. Kube-router implements allow and deny semantics through Kubernetes policies. Calico adds logging and pass actions; Antrea distinguishes drop and reject in addition to allow and pass. Cilium focuses on explicit allow and deny rules in its native policy resources.

Priority The final column indicates whether the CNI defines an explicit precedence model among policies. Calico resolves conflicts using policy order, and Antrea uses policy tiers. Kube-router and Cilium do not expose an equivalent ordering mechanism

in the same way; in Cilium, explicit deny rules take precedence over allow rules for the same traffic path.

Overall, the table shows that policy support varies significantly across CNIs. Flannel is unsuitable for policy verification because it does not enforce policies at all. Kube-router supports only namespace-scoped Kubernetes policies at layers 3 and 4. Calico, Cilium, and Antrea offer richer policy models, including cluster-wide scope and layer 7 controls.

3 Network Verification Approaches

According to the 2025 CNCF annual survey, security (36%) and complexity (34%) were among the most frequently reported challenges with containers [8]. In Kubernetes, network policies (Section 2.3) are a central security mechanism: they restrict traffic between pods and define connectivity constraints at the workload level. As policy sets grow, however, so does the risk of misconfiguration. Rules may be incomplete, inconsistent, or difficult to reason about across namespaces and services [5, 33, 34], exposing unintended communication paths and enabling lateral movement between workloads [5]. Prior work also shows that large policy sets may contain redundant or superseded rules that increase management overhead without changing effective connectivity [34].

These challenges are addressed by research on *network policy verification*. Two broad approaches have emerged. *Control-plane* methods analyze configuration and routing logic to infer the forwarding behaviour that would result from policy and protocol interactions [2, 12, 15, 16]. *Data-plane* methods, by contrast, analyze forwarding state, or a representation derived from it, to check properties such as reachability and isolation [19, 21, 35, 48]. The following sections review both approaches and discuss representative solutions from the literature. Although much of this literature was developed for traditional IP networks and software-defined networking (SDN) environments, it addresses the same fundamental questions that arise in Kubernetes.

3.1 Control Plane Approaches

Minesweeper Minesweeper [2] is an open-source network verification tool designed to proactively catch configuration errors before they can cause network outages or security vulnerabilities. Instead of running simulations or waiting for a network to converge, it treats the network control plane as a system of simultaneous constraints. This allows it to convert the complex, dynamic problem of network verification into a static math problem and verify how a network will behave across all possible environments, including different link-failure scenarios or external routing announcements.

To achieve this constraint-based verification, Minesweeper processes configurations through three distinct structural components:

- **Logical Formula Translation Engine:** This initial component translates low-level router configuration files into a unified logical formula. Instead of generating step-by-step simulations, this engine mathematically constructs a model of simultaneous constraints that natively captures the stable routing states emerging from interacting protocols like OSPF, BGP, and static routes.
- **Constraint Binder & SMT Infrastructure:** This component handles the combinational search by binding the generated network constraint model directly with the operator’s desired network properties into a single formula. Rather than explicitly computing vast message sets, it packages these simultaneous

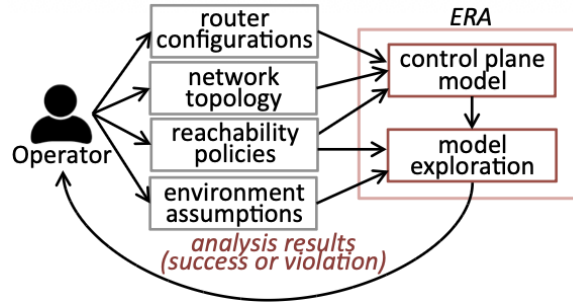


Figure 7: High level vision of ERA [12]

equations and hands them to an Satisfiability Modulo Theories (SMT) solver to execute a targeted mathematical search for a single counterexample.

- **Satisfying Assignment Resolver:** This final component evaluates the outcome of the static math problem solved by the SMT infrastructure. If it finds a satisfying assignment to the simultaneous constraints, it resolves this data as a stable state where the network policy is violated, pinpointing a bug. If it finds the formula to be unsatisfiable, it mathematically proves that the property holds true across all targeted scenarios.

ERA ERA [12] is a control plane verification tool that differentiates itself by executing an analytical approach called reachability analysis via reverse route propagation, as illustrated in Figure 7. Instead of modeling packets traveling forward through a network, it tracks how routing advertisements propagate in reverse relative to traffic trajectories to compute reachability. ERA processes configurations through three distinct internal processing stages:

- **Abstract Route Modeling & Visibility Function:** This initial stage inputs raw network configurations (parsed via an external tool, Batfish [15]) and converts them into an abstract mathematical representation. It unifies low-level protocols into 128-bit abstract routes that capture critical attributes (like destination prefixes and administrative distances) while stripping away unnecessary message details. It then applies a control plane visibility function to these routes, mathematically capturing how configurations filter, redistribute, and aggregate route advertisements across the entire network topology.
- **Symbolic BDD Processing Engine:** This stage handles the vast number of potential network environments by encoding sets of inputs and outputs using Binary Decision Diagrams (BDDs). Instead of computing massive lists of routing messages one by one, it maps the network’s behavior using symbolic BDD logic. By accumulating constraints across the network graph, a single execution over a symbolic BDD input allows ERA to evaluate an entire class of concrete environmental states simultaneously.

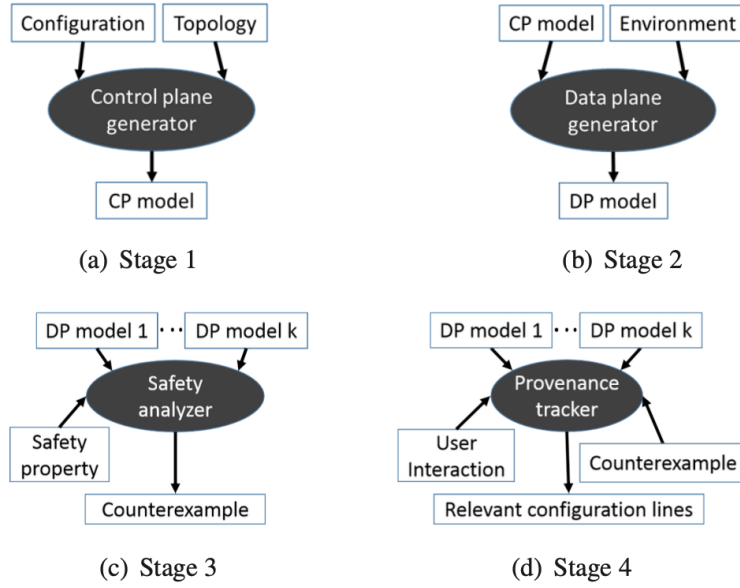


Figure 8: 4 stages of Batfish [15]

- **Vectorized Set Operation Resolver:** This final stage optimizes and resolves the math problem to output reachability results in near-real-time. It groups routes treated identically by the network into integer-indexed equivalence classes (atomic predicates). This converts complex boolean equations into simple set operations (unions and intersections), allowing ERA to use vectorized Central Processing Unit (CPU) instructions (Intel AVX2) to perform lightning-fast parallelized bit-wise OR and AND calculations directly on the processor.

Batfish Batfish [15] is a static analysis tool designed to find network configuration errors proactively, before they are deployed to a live network. It uses a fully declarative approach, translating network configurations and routing protocols into LogiQL (a variant of Datalog) to simulate how the network will behave. It is composed of 4 stages as illustrated in Figure 8:

- **Stage 1-Control Plane Generator:** Parses vendor-specific configuration files (Cisco, Juniper, Arista, etc.) and the network topology into a vendor-agnostic representation. It uses logic rules to capture how routing protocols collectively compute paths.
- **Stage 2-Data Plane Generator:** Takes an "environment" input (e.g., specific link failures or external route announcements) and executes the Datalog program. This computes a fixed-point model of the Routing Information Base (RIB) and FIB for each node.
- **Stage 3-Safety Analyzer:** Translates the generated data plane and desired correctness properties into constraints checked by the Z3/NoD constraint solver. If a property is violated, it generates a concrete counterexample packet.

- **Stage 4-Provenance Tracker:** Logs the counterexample packet back through the logical model, showing operators the exact path, ACL rules encountered, and the precise configuration lines responsible for the error.

ARC Abstract Representation for Control planes (ARC) translates configurations into a series of routing protocol-independent weighted digraphs, transforming complex network verification tasks into fast, polynomial-time graph problems. It processes configurations through three distinct internal processing stages:

- **Extended Topology Graph (ETG) Construction:** This initial stage extracts configuration directives (parsed via an external tool, Batfish) to model the network as an Extended Topology Graph (ETG). For every distinct traffic class (source-destination subnet pair), it creates a customized digraph where vertices represent individual routing processes (with separate input and output nodes) and directed edges represent the physical hardware or internal software channels over which data traffic can flow.
- **Protocol-to-Weight Currency Mapping:** This stage assigns mathematical weights to the ETG's directed edges to unify the different "currencies" used by various protocols (such as OSPF metrics, RIP hop counts, and BGP AS path lengths). It scales and combines link cost metrics, administrative distances (AD), and fixed route redistribution costs so that the minimum-cost path calculated on the graph perfectly matches the actual forwarding path preferred by the real network.
- **Graph-Level Invariant Evaluation:** This final stage runs polynomial-time graph algorithms over the generated ARCs to determine if intended network properties always hold true under arbitrary failures. For instance, it evaluates reachability and blocking by checking connected components, tracks firewalls via waypoint node removal, and validates multi-link fault tolerance by calculating max-flow/min-cut edge-disjoint paths.

3.2 Data Plane Approaches

Header Space Analysis Header Space Analysis (HSA) uses an analytical approach called protocol-agnostic geometric modeling. It treats the entire packet header as a flat concatenation of bits without any associated protocol meaning to evaluate network state. HSA processes configurations and analyzes network behavior through three distinct internal processing stages:

- **Geometric Header & Network Space Abstraction:** This initial stage inputs raw network data (parsed from physical device configuration files, medium access control (MAC) tables, ACLs, and routing tables) and converts the environment into a unified geometric space. It unifies all potential packet types into an L -dimensional space where each packet is a discrete point. By taking the cross-product of this bit-level header space with all physical switch ports, it

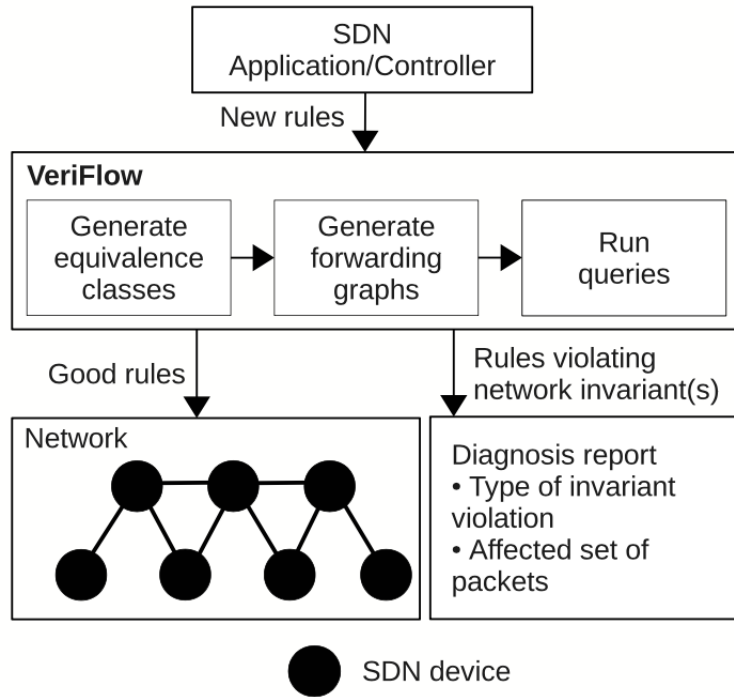


Figure 9: Architecture of VeriFlow [21]

constructs a comprehensive network space that maps precisely where and how traffic can exist within the topology.

- **Geometric Transfer Function Propagation Engine:** This stage models network hardware devices (routers, switches, firewalls, and middleboxes) and physical links as geometric transformers rather than protocol-dependent software processes. It captures data plane behavior using a network transfer function (Ψ) to model packet alterations and drops, and a topology transfer function (Γ) to model link traversal. By executing a breadth-first search on a propagation graph, it pushes symbolic "test packets" (wildcard expressions representing hypercubes) through these chained transformers to track how traffic flows split, rewrite, or drop.
- **Header Space Algebra & Range Inverse Resolver:** This final stage executes fundamental set operations (intersections, unions, and complements) on wildcard bit-expressions to check for network anomalies. It evaluates the final geometric blocks against universal invariants to detect reachability violations, forwarding loops, or cross-slice traffic leakage. When a violation is found, it computes the "range inverse" of the transfer functions along the path, working backward from the destination to output the exact set of matching packet headers that cause the failure.

VeriFlow VeriFlow uses an analytical approach called dynamic network slicing via incremental equivalence class tracking. Instead of checking the entire network

snapshot offline after changes occur, it sits as an active shim layer between the SDN controller and network devices to intercept and verify individual rule modifications before they hit the forwarding plane.

VeriFlow processes configurations and verifies data plane behavior through three distinct internal processing stages, as illustrated in Figure 9:

- **Generate Equivalence Classes:** This initial stage inputs raw rule updates intercepted via an active OpenFlow channel and slices the packet header space into discrete Equivalence Classes (ECs). It utilizes a multi-dimensional prefix tree (trie) where each dimension represents a specific packet header field. By traversing this trie dimension-by-dimension, VeriFlow isolates only the existing rules that intersect with the update, narrowing the analysis strictly to the small set of packet classes whose forwarding trajectories could actually be altered.
- **Generate Forwarding Graphs:** This stage models network data plane behavior by generating independent forwarding graphs for each modified EC. Nodes in the graph represent the EC at a distinct physical device, and directed edges model localized forwarding choices connecting one device to its next hop. To populate the graph quickly, it runs a targeted second traversal of the trie structure to capture any coarser background rules that influence the packet path, resulting in an exact, fine-grained representation of the active topology.
- **Run Queries:** This final stage executes pluggable verification functions directly on the constructed forwarding graphs to check for network anomalies in hundreds of microseconds. It runs optimized graph-search algorithms, such as depth-first search, over the isolated EC paths to detect invariant violations like basic reachability failures, forwarding loops, or policy bypasses. If a violation is flagged, it blocks the rule or generates an instant diagnosis report specifying the exact failure type and the affected packet sets.

Anteater Anteater [35] is an open-source data plane verification tool that differentiates itself by executing an analytical approach called static data plane analysis via boolean satisfiability (SAT) encoding. It diagnoses network state directly from the forwarding tables of routers, switches, and firewalls. Anteater processes data plane state and validates network invariants through three distinct internal processing stages:

- **Data Plane Collection & Boolean Modeling Engine:** This initial stage inputs raw Forwarding Information Bases (FIBs), topologies, and access control lists (ACLs) from network devices. It converts these forwarding policies into symbolic boolean functions over a global data graph, where a packet is modeled as an array of binary variables representing header fields.
- **LLVM IR Constraint Graph Synthesizer:** This stage compiles the abstract policies and high-level invariants such as loop-freeness, reachability, or consistency into the open intermediate representation (IR) of Low Level Virtual

Machine (LLVM). This converts the topology into a control flow graph, allowing standard compiler optimizations (like function inlining and expression simplification) to minimize the boolean logic and enforce constraints for packet transformations.

- **Vectorized SAT Formula Solver:** This final stage links the optimized LLVM IR code, parallelizes execution across independent router targets, and translates the finalized assertions into SAT formulas. It feeds these equations into an off-the-shelf SAT solver to mathematically resolve network states and output an explicit counterexample path and packet header if an invariant violation is found.

Libra Libra [48] is a highly scalable static verification tool that utilizes the MapReduce framework to check the correctness of forwarding tables in massive data center networks. Functionally, it uses the parallel processing power of MapReduce to divide a network's millions of routing rules into subnet-specific slices, allowing it to rapidly detect data plane errors

Libra is comprised of two main software components: a route dumper and a MapReduce-based route checker. The route dumper monitors timestamped event streams from network controllers to capture globally stable snapshots of the forwarding state and stores them in a distributed file system. The route checker then executes the verification process in three distinct steps:

- **The Map Phase:** Distributed mapper nodes load the full list of network subnets into an in-memory binary trie and ingest shards of the forwarding rules, matching each rule against the subnets to determine which rules affect which paths.
- **The Shuffle Phase:** The framework groups these matched pairs by individual subnet prefixes to isolate each subnet's relevant configuration into a distinct data "slice".
- **The Reduce Phase:** Reducer nodes receive these slices, resolve rule priorities to determine the actual paths packets will take, build a directed forwarding graph for each subnet, and run standard graph algorithms (such as Tarjan's algorithm for loop detection) to instantly surface routing anomalies.

3.3 Comparison

Control-plane and data-plane approaches differ fundamentally in what they analyze and when verification takes place. Control-plane tools reason about configuration and routing logic to predict the forwarding behaviour that would result after deployment. Data-plane tools, by contrast, analyze installed or live forwarding state, either from collected snapshots or during rule updates in a running network. Control-plane verification is therefore typically *proactive*: it can detect misconfigurations before they affect traffic. Data-plane verification is typically *reactive*: violations are identified only after forwarding rules have been produced and applied. A second difference lies in

diagnosability. Because control-plane tools operate closer to the original configuration, they can often relate a violation back to the policy or routing rule that caused it. Data-plane tools work on lower-level forwarding tables, so explaining why a violation occurred may require additional mapping back to high-level configuration. A third difference concerns input assumptions. Control-plane methods must model diverse routing protocols and vendor-specific configuration languages, whereas data-plane methods depend on accurate forwarding snapshots or live network integration but operate over a more uniform low-level representation.

However, significant trade-offs also exist *within* each family, as no single solution is universally optimal. Within control-plane approaches, Minesweeper [2] offers high expressiveness through SMT-based constraint solving and can verify properties across many possible environments but solver cost can become a scalability bottleneck as network size grows. Batfish [15] provides strong diagnostic provenance by tracing violations back to configuration lines but its multi-stage pipeline is heavier than more abstract alternatives. By contrast, ERA [12] and ARC [16] trade away some fine-grained detail for symbolic and graph-based abstractions that support faster reachability analysis at scale. A similar tension appears in the data-plane family. Header Space Analysis [19] offers granular, protocol-agnostic geometric modelling but analysis cost can grow quickly in complex networks with many rules and middleboxes. Libra [48] addresses scale through a MapReduce-based architecture well suited to massive data-center rule checking but depends on stable forwarding snapshots and batch analysis rather than interactive policy authoring. VeriFlow [21] provides low-latency, inline verification in SDN environments but requires live controller integration and is not designed for offline analysis of static policy manifests. Anteater [35] performs powerful static checking over collected forwarding state but SAT-based analysis does not always scale gracefully.

To address the challenges of Kubernetes network security, this thesis adopts a control-plane verification approach. Kubernetes security policies are defined declaratively as YAML manifests together with application topology files, before workloads are deployed. Relying on data-plane verification would require policies to be translated into low-level forwarding rules such as `iptables` or `eBPF` programs and distributed across live nodes before inconsistencies could be detected. By analyzing policy structure offline at the control-plane level, this work supports proactive validation of configurations before they reach a running cluster. This approach is also better aligned with cloud-native operation: verification is decoupled from the volatile life cycle of ephemeral pods and does not depend on parsing the internal data-plane implementation of a specific CNI plugin. The tool developed in this thesis therefore provides a scalable, vendor-agnostic, and preventative method for analyzing Kubernetes and Cilium network policies over a fixed application topology.

4 Methodology

This chapter presents the research methodology adopted in this thesis. Based on the problems identified in Section 1, we detail how the proposed solution addresses these problems, explain its practical application, and describe the methods used to evaluate its correctness and scalability.

4.1 Research Approach and Proposed Solution

This thesis is motivated by four related problems in Kubernetes network policy management. First, it is difficult to understand effective communication in a cluster. Second, clusters may contain heterogeneous policy types, including allow and deny rules, as well as policies expressed in different CNI formats. Third, it is hard to determine how individual policies contribute to the overall policy outcome when multiple policies apply to the same workload. Fourth, operators lack systematic support for comparing policy sets, for example to decide whether two configurations produce the same connectivity, or whether one set contains rules that are redundant with respect to another.

To address these challenges, we adopt an offline control-plane approach. The proposed framework analyses application and network policy manifests directly, without requiring a running cluster or live traffic. From these inputs, it models effective cluster communication using a reachability matrix. This matrix empowers operators to inspect the overall communication landscape, verify connectivity between specific pairs, and compare different policy sets to determine whether they yield the same outcome or whether one set contains redundant rules.

The proposed approach is designed to be agnostic to policy format and CNI implementation. Network policies are normalized into a common internal representation, regardless of whether they are expressed as native Kubernetes `NetworkPolicy` resources or as CNI-specific custom resources. The connectivity model therefore depends on the interpreted meaning of those policy rules after normalization, not on a single CNI plugin.

The model represents communication at multiple levels of granularity, and its rows and columns are not symmetric. Sources (rows) represent the origin of a connection. A source is either a namespace or a workload derived from application manifests, such as a Pod, Deployment, StatefulSet, ReplicaSet, or Job. Sources do not encode port or protocol, because Kubernetes and Cilium network policies restrict destination ports rather than the originating port of the sender.

Destinations (columns) represent the target of a connection. A destination may be a namespace, a workload, a Service, or an external CIDR from an `ipBlock` rule, and it may additionally include port and protocol when the policy specifies them. Because a Service cannot initiate communication, it is considered only as a destination. This asymmetry follows from how network policies are expressed in Kubernetes and CNI implementations. Policies identify affected workloads and permitted peers through pod and namespace selectors, while port and protocol constraints apply to the destination of a connection rather than to the source.

Both sources and destinations are encoded in the common form `namespace_-workload_port_protocol`, where unused fields are replaced by the wildcard `*`. For source rows, the port and protocol fields are always wildcards. For destination columns, any field may be concrete or wildcard, depending on how specific the policy rule is. Matrix cells take only two values: 1 means that the corresponding source–destination communication is allowed under the analyzed policy set, and 0 means that it is denied. The matrix is initialized to allow at namespace level, reflecting Kubernetes’ default-open pod network, and is then refined by policy application. At query time, auxiliary policy metadata is also used so that Cilium deny, policy allow, policy deny, and default connectivity can be distinguished even though the stored cell values remain 0 or 1.

Reachability queries ask whether a concrete communication is permitted. A query may specify source namespace and optionally source workload, together with destination namespace and optionally destination workload, port, and protocol. Evaluation first collects all matrix rows and columns that match these parameters, including wildcard endpoints, and then resolves the matching cells by priority tier to yield a single allow or deny answer. If no matching row or column exists, the query is invalid.

The solution creates this model from two inputs: application manifests and network policy manifests. Application manifests describe the cluster topology, including namespaces, workloads, services, and declared ports, while policy manifests describe ingress and egress allow and deny rules. From these inputs, it provides three capabilities. The **reachability matrix** encodes allowed and denied communication between cluster endpoints and makes the effective policy outcome visible across the application topology. **Connectivity queries** answer whether a specific source–destination communication is permitted at namespace, workload, port, or protocol level. **Policy comparison** enables operators to compare two policy sets over the same topology and detect equivalence, redundancy, and other relationships between policies, including policies expressed in different formats.

4.2 Evaluation Design

Evaluation objectives The evaluation assesses whether the three capabilities above behave correctly and remain practical as policy count grows. Specifically, it addresses four core questions:

- **Matrix accuracy:** Is the computed reachability matrix correct?
- **Query correctness:** Do connectivity queries return the correct allow or deny decisions?
- **Simplifier validity:** Does the policy simplifier accurately identify semantically equivalent versus non-equivalent policy sets?
- **Scalability:** Does the approach remain practical as the number of policies increases?

Testbed Applications To evaluate the framework on realistic topologies, we employ two public sample applications commonly adopted in Kubernetes and microservices research. The first, `aks-store-demo`, is Microsoft’s sample e-commerce application [36], which comprises multiple cooperating services including a storefront, order processing, and database components. While the official manifests deploy all workloads within a single namespace, we adapt this topology by partitioning the services into `frontend-ns`, `backend-ns`, and `database-ns` to exercise namespace-level policy reasoning. This application provides a diverse set of workloads and declared service ports, and the manifests used in this thesis are derived directly from the official repository. The second application, `istio-bookinfo`, is the standard benchmark from the Istio project [18]. Similar to the store demo, its default deployment runs all services within a single namespace; for our evaluation, we redistribute them across three distinct namespaces. Both applications are well-established benchmarks, having been utilized in prior literature [3, 11, 20, 42, 40]. These adapted topologies are highly representative of production environments, as they incorporate multiple namespaces, labelled workloads, complex inter-service dependencies, and realistic port exposures.

Policy scenarios For each application, we define policy scenarios intended to exercise the main semantic features supported by the tool, including pod and namespace label selectors, ingress and egress rules, explicit ports and port ranges, `ipBlock/CIDR` rules, allow-all and deny-all baselines, and CNI-specific deny and cluster-wide policies. The scenarios are not intended to enumerate every possible policy but to cover the constructs implemented by the tool and the edge cases most likely to affect policy reasoning, such as multiple policies applying to the same workload, policies selecting more than one peer workload, namespace-level endpoints, external IP blocks, and interactions between default-allow and explicit deny rules.

Test environment All automated correctness and performance experiments were executed locally on an Apple M2 laptop with 16 GB Random Access Memory (RAM), running macOS 26.0.1. This local setup was chosen for ease of iteration. Its implications, including whether the observed runtimes are representative of an operator running the tool on a workstation rather than inside a cluster management environment, are discussed in Section 7.3.

Reachability matrix tests Correctness is evaluated by comparing tool output against manually derived reference results. For reachability matrix tests, the ground truth was produced by tracing each policy scenario by hand against the corresponding application topology and recording expected allowed and denied source–destination pairs. A test passes only when the row labels, column labels, and cell values all match this reference matrix. This verifies that policy parsing, endpoint modelling, ingress/egress composition, and deny-rule handling are implemented correctly.

Connectivity query tests For connectivity query tests, the ground truth is derived from the same manual analysis. The query suite covers the main query shapes

supported by the tool, including pod-to-pod queries with explicit port and protocol, namespace-to-namespace queries without workload or port constraints, namespace-to-workload queries, and ipBlock/CIDR-related cases, including invalid queries where no matching matrix endpoint exists. Each query case is paired with a policy scenario and an expected allow/deny decision.

Policy simplifier tests For policy simplifier tests, we use three comparison types per application because they exercise distinct comparison goals. The **non-equivalent** case pairs two policy sets that govern the same source–destination flow differently, testing whether the simplifier correctly reports a mismatch. The **superset-redundant** case contains an additional rule that does not change effective communication, testing redundancy detection. The success criterion is that every defined test case passes. A pass means that the tool behaves as expected on the scenario suite.

Performance evaluation The goal of the performance evaluation is to assess the scalability limits of the current implementation and estimate the resource requirements of the approach in larger deployments. We measure behaviour under synthetic policy loads of 10, 25, 50, 100, and 1000 policies, representing a small, medium, and large policy set that stresses matrix construction and comparison cost. The synthetic policies are generated programmatically from each application’s workload list: each policy selects a source workload and a peer workload using label selectors, alternates between ingress and egress rules, and optionally attaches a Transmission Control Protocol (TCP) port. For each load level, we record matrix construction time to measure how long it takes to build the reachability model, matrix size to quantify the footprint of the ingress, egress, and final reachability matrices, query latency to assess whether connectivity lookups remain responsive once the matrix has been built, and policy simplifier runtime to measure the end-to-end cost of building two reachability models and comparing them. The measured results are reported in Section 6.

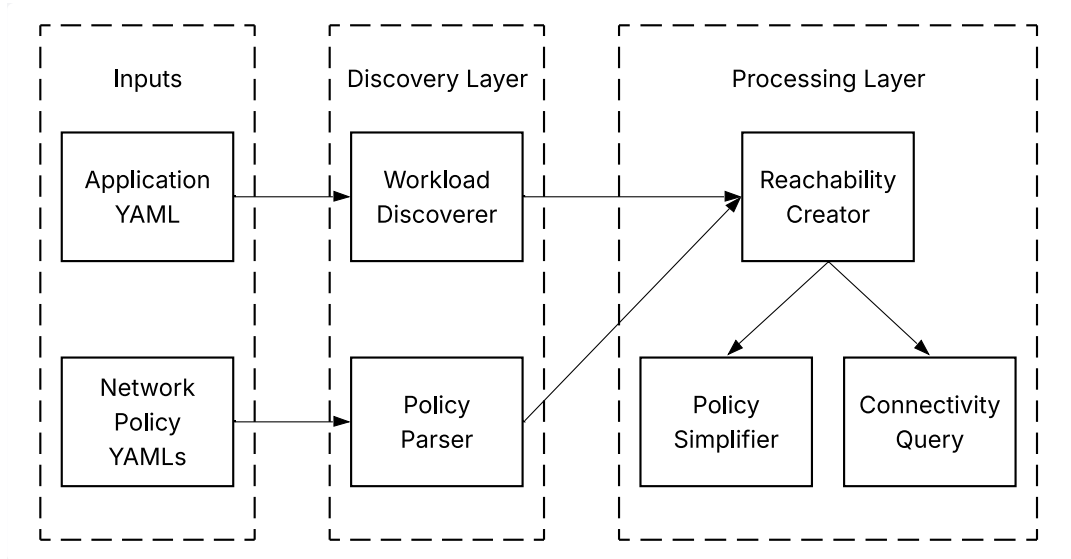


Figure 10: The Architecture of the tool

5 Design and Implementation

This chapter describes the design choices and implementation details of the verification tool. All components are implemented in Python.

Figure 10 shows the end-to-end architecture of the tool, organized into an input stage, a discovery layer, and a processing layer. The process begins with the input files: application YAML manifests that describe the cluster topology, and network policy YAML manifests that define the traffic rules applied to that topology. These inputs are processed in the *discovery layer* by two components. The **Workload Discoverer** parses the application manifests and extracts the information required for modeling, including workload labels, namespaces, container ports, and service selectors. The **Policy Parser** reads the network policy definitions and normalizes them into a common internal representation, capturing source labels, target labels, namespace scope, ports, and policy direction (ingress and egress). The outputs of both parsers are passed to the *processing layer*. The **Reachability Creator** combines the discovered topology with the parsed policies and constructs a reachability matrix that represents allowed and denied traffic between cluster endpoints. From this matrix, the tool supports **connectivity queries**, which answer whether a given source can reach a given destination under the current policy configuration. Finally, the **Policy Simplifier** compares different policy configurations by reusing the Reachability Creator to detect equivalent policy states, redundant rules, and supersets across syntactically different policies.

The remainder of this chapter follows the order shown in Figure 10. Section 5.1 introduces the discovery layer and the processing of application and network policy input files by the Workload Discoverer and Policy Parser. Section 5.2 covers the creation of the reachability matrix from the parsed inputs. Section 5.3 addresses the resolution of connectivity queries against that matrix. Finally, Section 5.4 covers

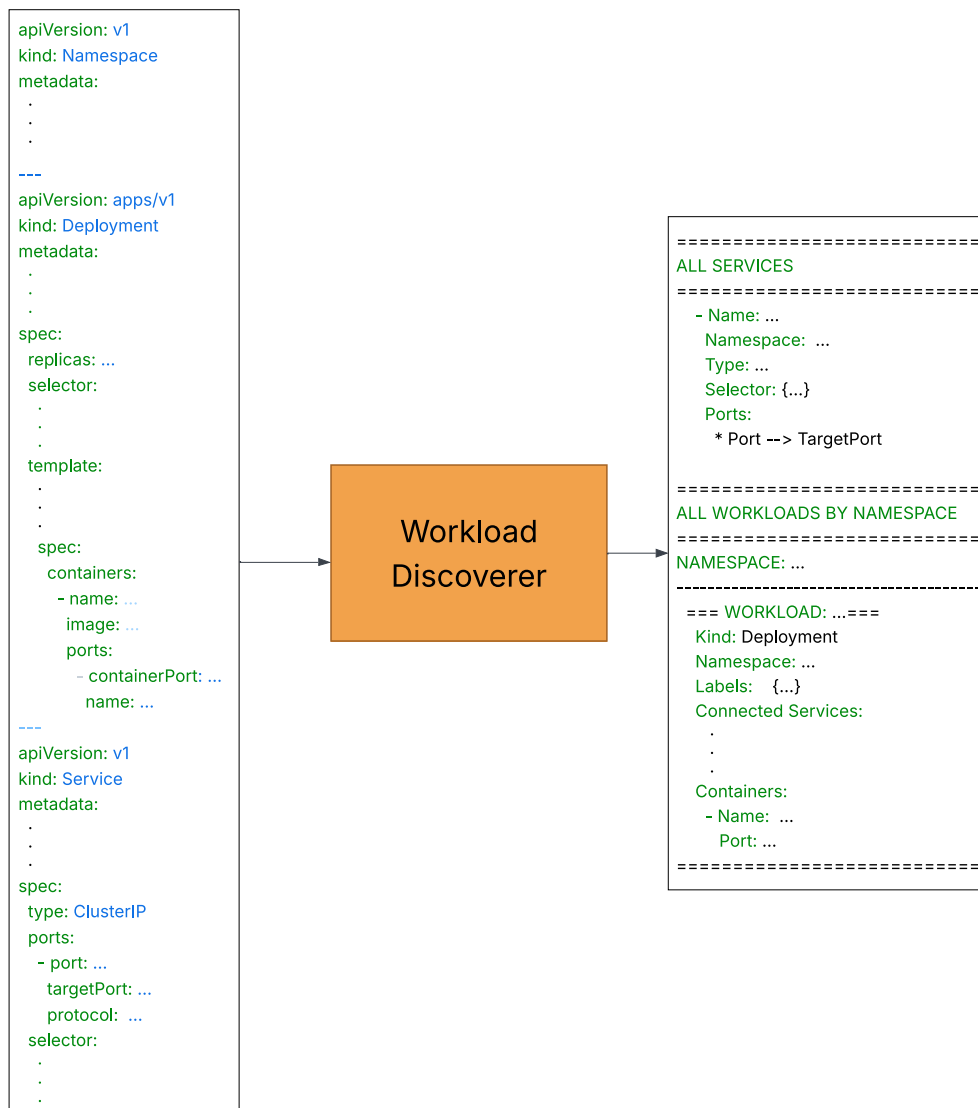


Figure 11: Input and output of the Workload Discoverer

policy comparison and simplification.

5.1 Workload and Policy Parser

Workload Discoverer Discovering the application topology is a crucial step, because the network state is derived by matching application components against network policies. As shown in Figure 11, the Workload Discoverer takes Kubernetes application YAML manifests as input and produces a list of namespaces, together with the corresponding workloads, services, containers, and ports. The discoverer assumes that the input YAML is syntactically valid and that it accurately reflects the open ports and application components; the implications of this assumption are discussed in

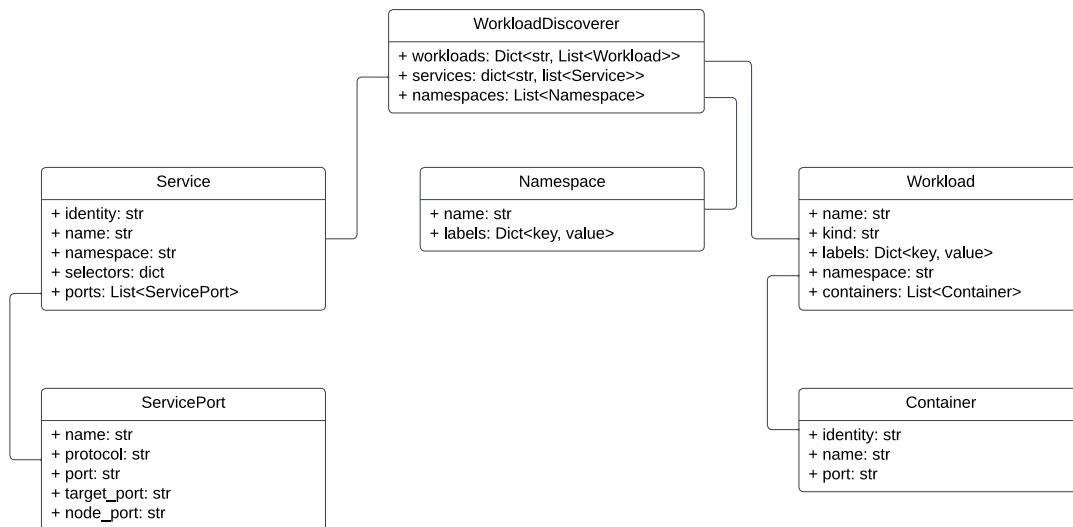


Figure 12: UML class diagram of the Workload Discoverer

Section 7.3.

The discoverer supports several Kubernetes resource kinds used to build the reachability model: **Pod**, **Deployment**, **ReplicaSet**, **StatefulSet**, **Job**, **Service**, and **Namespace**. Because network policies are evaluated through label selectors, the discoverer organizes workloads and services by namespace and stores component labels in Python dictionary which supports efficient matching, as illustrated in Figure 12. Grouping by namespace prevents workloads with the same name in different namespaces from being mixed up and allows all components in a given namespace to be retrieved without scanning the entire cluster topology.

Network policies can also select namespaces through `namespaceSelector`, which matches namespace labels rather than namespace names alone. Application manifests do not always define an explicit `Namespace` object, and when they do, it may contain no custom labels. In Kubernetes, every namespace is nevertheless associated with the standard label `kubernetes.io/metadata.name`, whose value is the namespace name. To preserve this behaviour, the discoverer creates an implicit namespace entry whenever a namespace is encountered only through other resources, or when a `Namespace` object defines no labels. This entry uses the standard `kubernetes.io/metadata.name` label set to the namespace name, so later matching against `namespaceSelector` expressions remains correct.

Policy Parser The Policy Parser reads network policies expressed as Kubernetes network policies and CRDs. Cilium is explicitly supported because it is among the most among the most widely used adopted CNI plugins, as discussed in Section 2.4. Rather than building separate analysis path for each policy language, the parser normalizes both kinds into a common internal representation, as illustrated in Figure 14. This allows the Reachability Creator to apply the same matching logic regardless or whether a rule originated in Kubernetes or Cilium syntax. The parser supports the three native Kubernetes selectors types as `podSelector`, `namespaceSelector`, and `ipBlock`

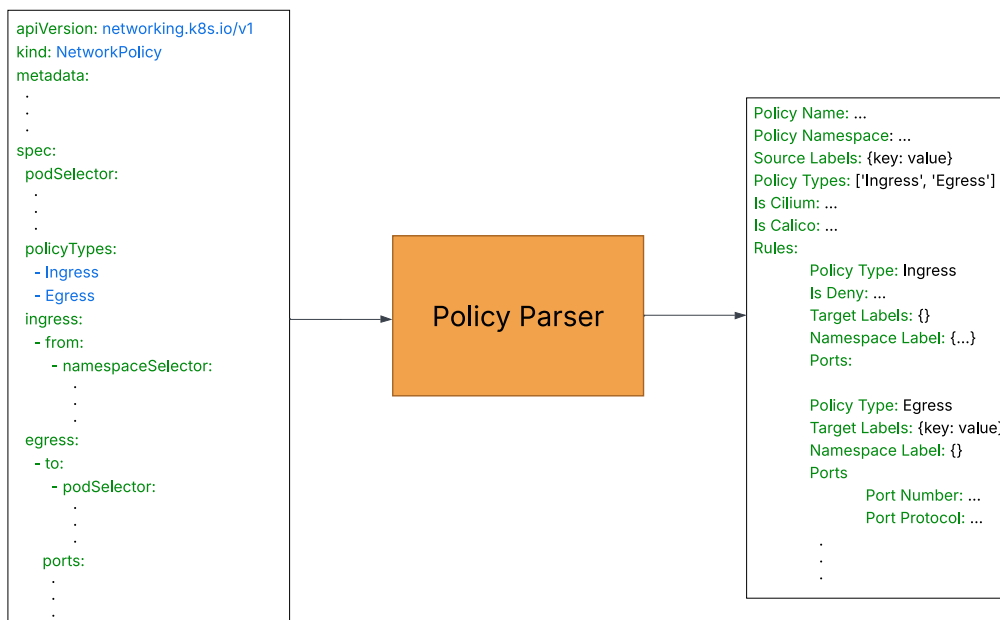


Figure 13: Input and output of the Policy Parser

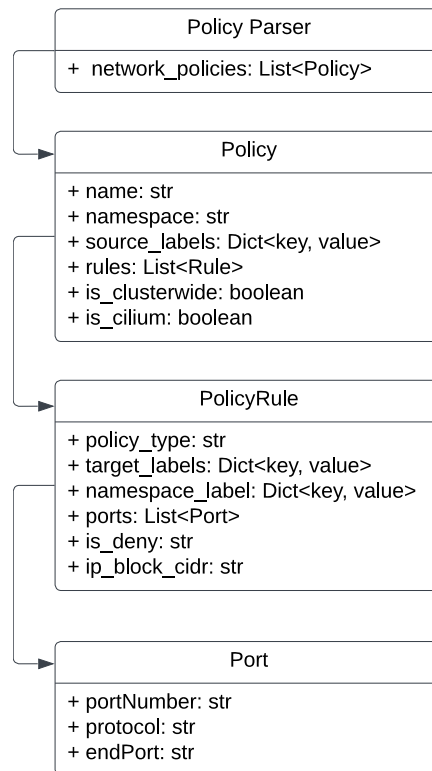


Figure 14: UML diagram of the Policy Parser

and assumes that the input manifests are syntactically valid. It produces a structured list of policies and their associated rules, as illustrated in Figure 13.

A native Kubernetes `NetworkPolicy` is a namespaced resource. It is bound to the namespace given in `metadata.namespace` and cannot be applied to multiple namespaces. Cross-namespace effects therefore arise only from rule selectors. For example, an ingress or egress rule may use `namespaceSelector` to target pods in another namespace. Cilium extends this model with `CiliumClusterwideNetworkPolicy`, which can apply cluster-wide. The parser therefore records the namespace scope of each policy and the namespace constraints attached to individual rules when a `namespaceSelector` is present. These constraints are extracted from the `namespaceSelector` expression in Kubernetes and from the `k8s:io.kubernetes.pod.namespace` expression in Cilium.

Once the namespace scope has been determined, the parser extracts the traffic direction. For Kubernetes `NetworkPolicy` resources, direction is expressed through `policyTypes`. If `policyTypes` is omitted, `Ingress` is assumed by default, and `Egress` is included only when the manifest defines egress rules. Cilium policies follow a different convention: the regulated directions are inferred from the rule sections present in the CRD (`ingress`, `egress`, `ingressDeny`, and `egressDeny`). Rules whose direction is not active for a given policy are excluded during parsing, so that only semantically relevant rules are passed to the Reachability Creator.

Kubernetes and Cilium also differ in how selectors are expressed. In Kubernetes `NetworkPolicy`, ingress rules are defined under `from` and egress rules under `to`;

Concept	Kubernetes	Cilium
Target pods	podSelector	endpointSelector
Ingress rules	from	fromEndpoints
Egress rules	to	toEndpoints
Namespace constraints	namespaceSelector	namespace labels in endpointSelector
IP address	ipBlock	fromCIDR / toCIDR

Table 3: Correspondence between Kubernetes and Cilium network policy selectors



Figure 15: Example of equivalent Kubernetes and Cilium policies normalized into the same internal representation

Cilium uses `fromEndpoints` and `toEndpoints`. Kubernetes selects target pods through `podSelector`, whereas Cilium uses `endpointSelector`, which may also encode namespace constraints. IP addresses are referenced through `ipBlock` in Kubernetes and through `fromCIDR` / `toCIDR` in Cilium. Table 3 summarizes this correspondence. Despite these syntactic differences, equivalent policies are normalized into the same internal representation. Figure 15 shows an egress rule expressed in Kubernetes and Cilium that results in the same internal policy object. The parser maps these syntactic differences into the same internal rule model so that later processing does not need to distinguish between policy languages at the matrix level.

Two further cilium features depart from standard Kubernetes semantics and therefore represented explicitly during parsing. First, `CiliumClusterWideNetworkPolicy` applied cluster-wide rather than within a single namespace. Second, Cilium supports explicit deny rules (`ingressDeny` and `egressDeny`), which can override allow rules and therefore do not follow Kubernetes' additive, allow-only model. Recording whether a policy is cluster-wide and whether a rule is a deny rule preserves these semantic differences in the internal representation, so that the Reachability Creator can apply the correct policy ordering and priority later.

Port parsing also differs between the policy kinds. In Kubernetes `NetworkPolicy`, an unspecified protocol defaults to TCP. In Cilium, an unspecified protocol is treated as matching any Layer 4 protocol (e.g., TCP and User Datagram Protocol (UDP)) and is represented internally using the wildcard symbol (*). Preserving this distinction

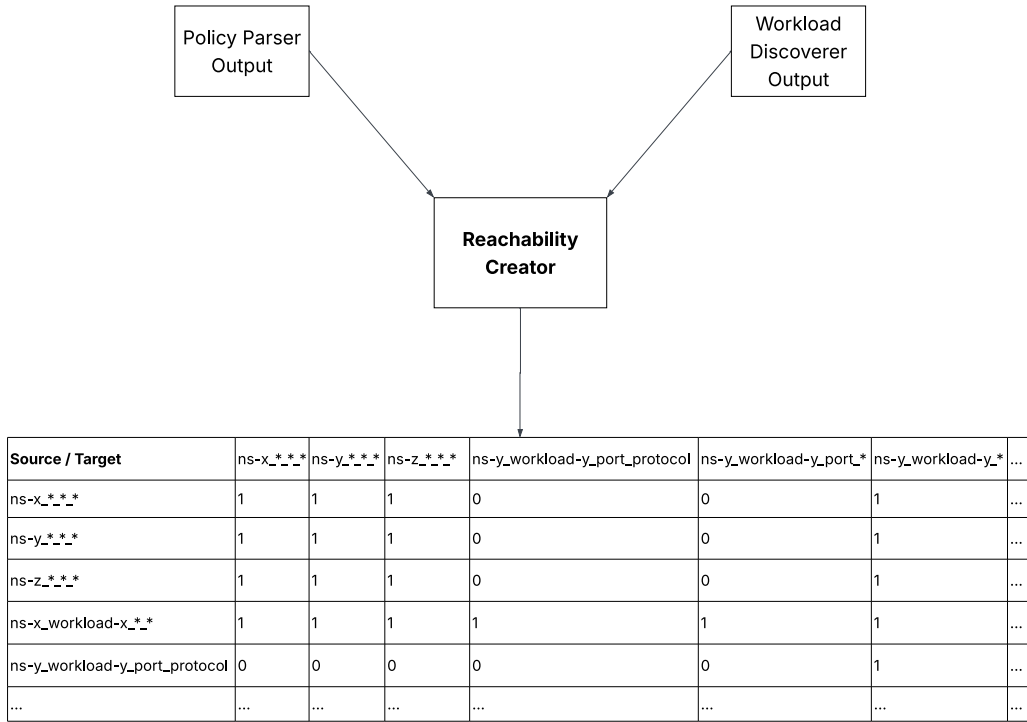


Figure 16: Input and output of Reachability Creator

during parsing avoids misrepresenting Cilium rules as TCP-only restrictions during matrix construction.

The remaining parser design is shared across both policy kinds. Source and target labels are stored as Python dictionaries to support efficient matching against workload and namespace labels during matrix construction. Rules are stored as lists because a single policy may contain multiple ingress or egress rules, and the ports defined in each rule are likewise stored as lists, since a rule may specify multiple ports. Port ranges are preserved through an optional `endPort` field in addition to `portNumber`, covering both Kubernetes and Cilium port-range syntax.

5.2 Reachability Matrix Creation

The Reachability Creator is the central component of the processing layer. It takes the parsed network policies together with the discovered workloads, services, and namespaces, and produces a reachability matrix that encodes allowed and denied traffic between cluster endpoints, as shown in Figure 16. The matrix contains only the sources and targets affected by the given network policies; connectivity involving other endpoints is resolved later through the connectivity queries described in Section 5.3.

Matrix Representation Both rows and columns follow the naming convention `namespace_workload_port_protocol`. The wildcard symbol `*` may replace any section and indicates that the corresponding component matches all values. It allows

the matrix to represent connectivity at different levels of detail, from namespace-wide rules to workload and port specific restrictions, without requiring every field to be fixed in advance. This flexibility is needed because network policies often specify only part of an endpoint and connectivity queries must still be resolved correctly when the user provides partial source or destination information. For example, the endpoint `namespace_*_port_protocol` matches every workload in the specified namespace for the given port and protocol. Underscores are used as separators because Kubernetes component names must comply with Request for Comments (RFC) 1123 [28], which does not permit underscores in resource names; using underscores as separators therefore avoids confusion between component names and endpoint structure. Row naming follows the same convention as columns, except that rows do not include port and protocol sections. Network policies regulate destination ports in their rules; neither Kubernetes nor Cilium network policies allow origin ports to be specified, so source rows represent workloads or namespaces rather than individual source ports. The matrix contains two values: 1 (allow) and 0 (deny). It is initialized with namespace-level endpoints in both rows and columns, all set to 1, representing full connectivity between namespaces before any policy is applied.

The matrix is not created with every possible workload, service, port, and protocol already present. Doing so would produce a large and mostly unused structure, so the matrix is kept sparse for efficiency. It is first initialized with only namespace-level endpoints in both rows and columns, all set to 1, representing full connectivity between namespaces before any policy is applied. During policy application, new rows and columns are added when a rule refers to a workload, service, port, protocol, or CIDR that is not yet in the matrix. In this way, the set of rows and columns grows incrementally with the policies being processed, rather than being fixed in advance.

Matrix Construction Kubernetes network policy semantics are additive and allow-only: once a pod is selected by a policy, it enters a restricted state and may communicate only through explicitly allow rules. Cilium additionally supports explicit deny rules that can override previously applied allow rules. To reflect these semantics faithfully, the Reachability Creator applied policies in three ordered phases. In the first phase, empty policies that declare `policyTypes` but contain no ingress or egress rules are processed. These policies implement implicit deny-all behaviour for the selected workloads. In the second phase, allow rules are applied according to Kubernetes' additive semantics. In the third phase, Cilium explicit deny rules are applied and may override earlier allow decisions. Each phase refines the result of the previous one so that the final matrix reflects the correct connectivity state.

Selector Matching Network policies do not affect the entire cluster uniformly; each rule applies only to the workloads and endpoints selected by its label expressions. The Reachability Creator must therefore determine which workloads are affected by each rule before updating the matrix, as illustrated in Figure 17. Although the same label-matching criteria are used in both cases, ingress and egress rules assign different roles to the selected endpoints. This is because both matrices represent sources as

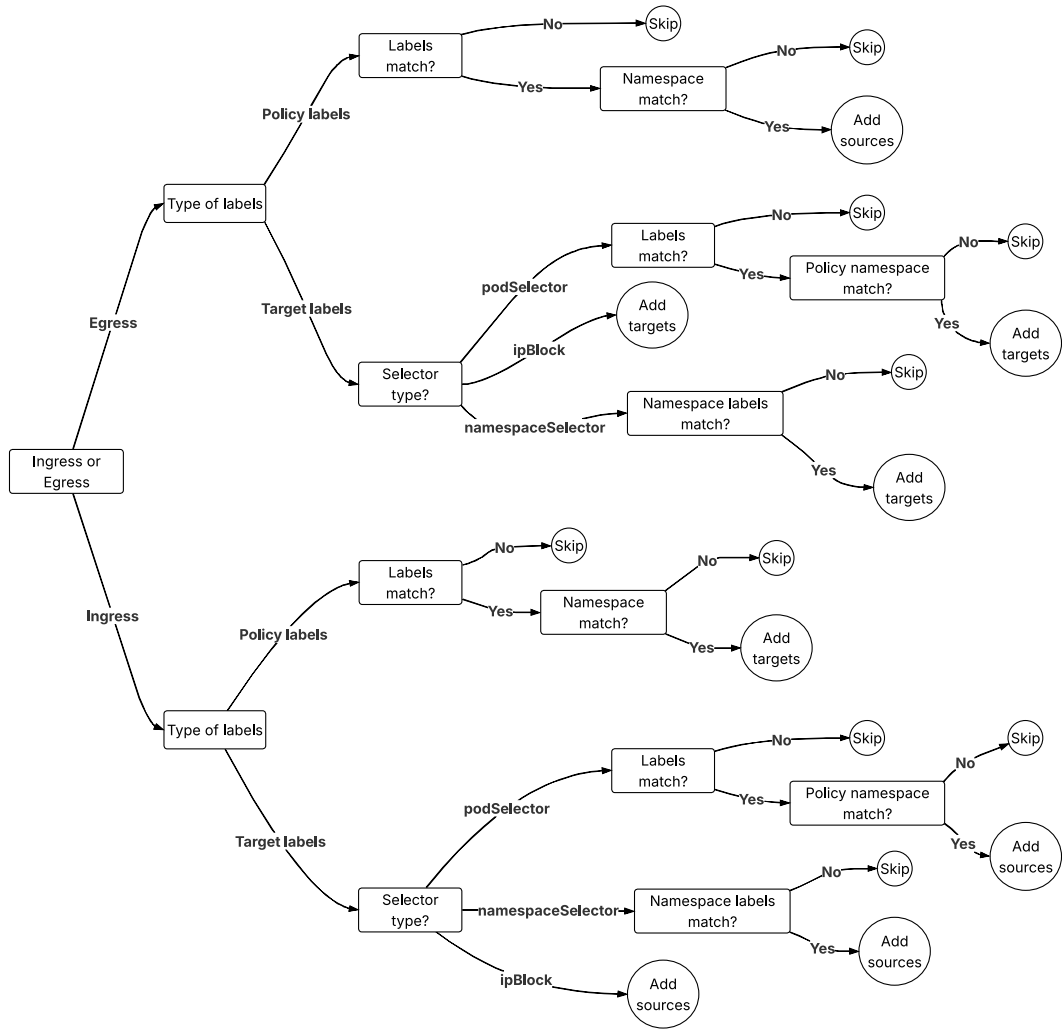


Figure 17: Decision tree of selector matching

Ingress Matrix				Egress Matrix				Reachability Matrix		
Source/Destination	dest1	dest2		Source/Destination	dest1	dest2		Source/Destination	dest1	dest2
src1	1	0	X	src1	1	1	=	src1	1	0
src2	1	1		src2	0	1		src2	0	1

Figure 18: Example of element-wise multiplication of the egress and ingress matrices

rows and destinations as columns, while ingress rules regulate who may send traffic to a protected destination and egress rules regulate where a restricted source may send traffic.

For both rule types, a workload matches a selector when every label in the selector is present on the workload. In addition, the workload namespace must match the policy namespace unless the policy is cluster-wide. For `ipBlock` rules, no workload matching is performed because control-plane analysis cannot determine workload IP addresses; external targets are therefore added directly to the matrix.

Egress rules In an egress rule, the policy `podSelector` identifies the **source** workloads, and the endpoints selected by `to / toEndpoints` identify the allowed **destinations**. Target selector matching follows three cases: with a `podSelector`, the selector labels must match the destination workload and the namespace constraint must be satisfied; with a `namespaceSelector`, the destination namespace must match the selected namespace labels; with an `ipBlock`, the external CIDR is added directly to the matrix.

Ingress rules In an ingress rule, this mapping is reversed. The policy `podSelector` identifies the **destinations**, and the workloads selected by `from` and `fromEndpoints` identify the allowed **sources**. The same three target selector cases apply but they are used to select sources rather than destinations. When a rule combines multiple selectors, all conditions must be satisfied.

Ingress and Egress Matrices Kubernetes evaluates ingress and egress policies independently: an egress rule restricts what a pod may send, whereas an ingress rule restricts what a pod may receive. A connection is permitted only when it is allowed from both perspectives. For this reason, the Reachability Creator maintains two separate matrices during construction: an *ingress matrix* and an *egress matrix*. Egress rule update the egress matrix; ingress rules update the ingress matrix. Final reachability is computed by element-wise multiplication of the matrices, as shown in Figure 18, so that a flow must be permitted on both on egress and ingress sides.

The two matrices use a transposed mapping that reflects Kubernetes semantics. In an egress rule, the workloads selected by the policy’s pod selector define the restricted **sources** (rows), and the targets selected by `to / toEndpoints` define the allowed **destinations** (columns).

Egress rules: When the first egress policy is applied to a source workload, the entire corresponding row is set to deny and only the explicitly allowed destination endpoints in that rule are set to allow. This models Kubernetes’ default-deny behaviour for egress once a pod is selected by a policy. If the same source is affected by a later

Endpoint	Connectivity Value
namespace_workload_port_protocol	1 (allow)
namespace_workload_port_*	0 (deny)
namespace_workload_**	0 (deny)
namespace_***	unchanged

Figure 19: Example of wildcard aggregation for a pod-level rule

egress policy, the row is not reset; only the values for the newly referenced destinations are updated, preventing previously allowed connections from being overwritten. After the egress matrix is updated, any missing rows and columns are also added to the ingress matrix so that both matrices share the same endpoint names.

Ingress rules: When the first ingress policy is applied to a destination column, the corresponding column is set to deny, and only the allowed source workloads are set to allow. Subsequent ingress policies affecting the same destination update only the relevant source–destination cells. After the ingress matrix is updated, missing rows and columns are also added to the egress matrix so that both matrices remain compatible for later combination.

Wildcard Aggregation Network policies that specify a destination port and protocol permit traffic only on that exact combination. The reachability matrix, however, also contains broader destination endpoints in which one or more components are replaced by the wildcard symbol (*). These broader endpoints are needed so that connectivity queries can be resolved at different levels of specificity, for example when a query does not specify a port or protocol. Wildcard aggregation ensures that these less specific endpoints reflect the same restriction as the fully specified one. When a rule targets a specific workload, port, and protocol, the allowed destination is encoded as `namespace_workload_port_protocol`. Because the policy applies only to that workload and only on the given port and protocol, broader parent endpoints such as `namespace_workload_port_*` and `namespace_workload_**` are set to deny, as shown in Figure 19. Otherwise, a query to a less specific endpoint could incorrectly suggest that other protocols or ports are also permitted. Aggregation stops at the workload level and does not collapse further to `namespace_***` because the policy restricts one workload rather than the entire namespace. For namespace-level rules with a specific port and protocol, encoded as `namespace_**_port_protocol`, the workload component is already unspecified. In this case, aggregation also updates `namespace_**_port_*` and `namespace_***` because the restriction applies to the namespace as a whole rather than to a single workload.

Examples Figure 20 illustrates egress processing step by step. The example starts from an initial namespace-level matrix in which all cells are set to allow, representing full connectivity between namespaces before any policy is applied. The egress policy shown in the figure then selects `order-service` in `backend-ns` as the restricted source and permits traffic only to `makeline-service` on TCP

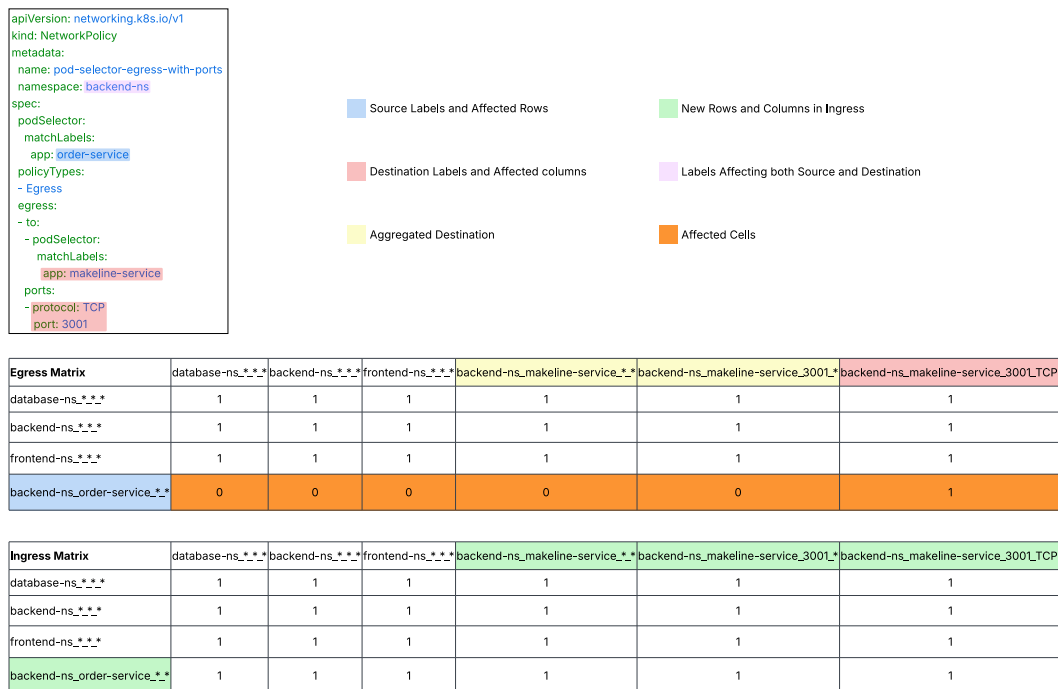


Figure 20: Example of egress policy application in the Reachability Creator

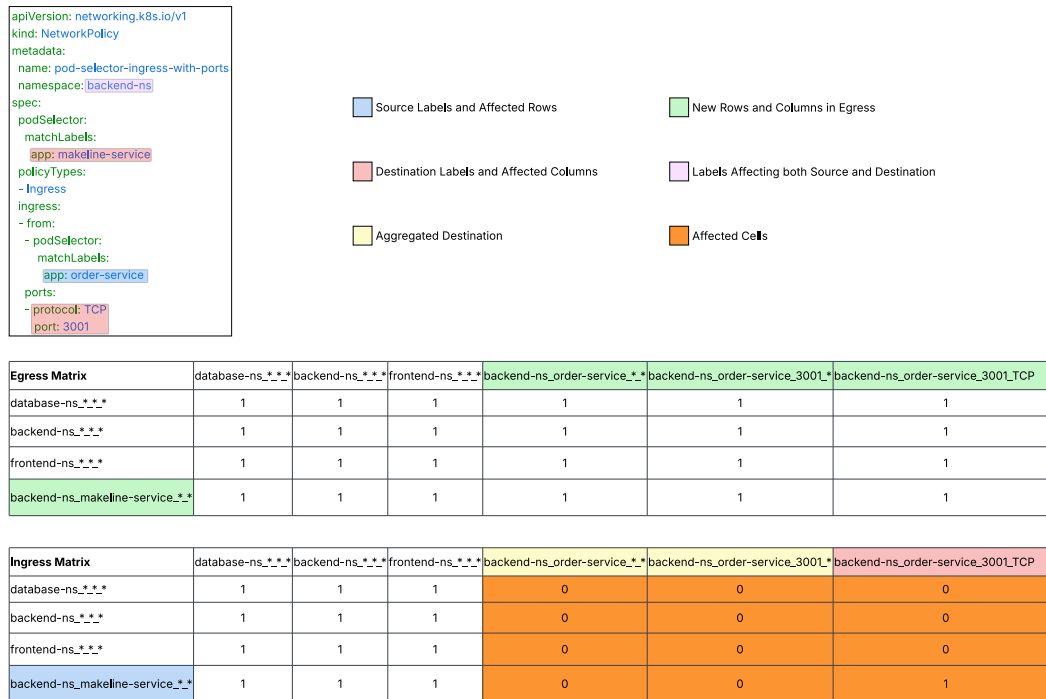


Figure 21: Example of ingress policy application in the Reachability Creator

port 3001. Because `order-service` is not yet present in the matrix, a new source row is added and the row is initialised to deny. A destination column for `backend-ns_makeline-service_3001_TCP` is then created and set to allow for that source. Since the selected pod is now policy-controlled, `order-service` cannot reach `makeline-service` on any less specific endpoint. Wildcard aggregation therefore introduces broader destination columns as `backend-ns_makeline-service_3001_*` and `backend-ns_makeline-service_*_*`, which are denied for `order-service` while remaining allowed for unrestricted sources. To keep ingress and egress matrices compatible for later combination, the same rows and columns are also added to the ingress matrix. Because no ingress policy is present in this example, those endpoints are inserted with allow values.

Figure 21 shows the corresponding ingress case, again starting from a fully connected namespace-level matrix. In the ingress matrix, the policy namespace together with the protected workload, protocol, and port select the destination column, while the allowed source workloads select the rows. For `makeline-service` on TCP port 3001, all values in the corresponding column are set to deny except in the `order-service` row, which is set to allow. Wildcard aggregation then denies broader destination columns for all sources, including `order-service` because traffic to less specific endpoints is not permitted by the rule. The same endpoints are mirrored into the egress matrix so that the two matrices can be combined later. Because no egress policy is present in this example, the new endpoints are inserted with allow values.

Cilium-Specific Cases The logic described above applies to all Kubernetes network policies and to most Cilium network policies. Two additional mechanisms are required for the remaining Cilium cases. The first case is cluster-wide scope. A `CiliumClusterwideNetworkPolicy` may match endpoints in every namespace but the matrix otherwise uses concrete namespace names in endpoint identifiers. To represent this behaviour, the namespace component of affected endpoints is replaced with the wildcard `*` whenever a cluster-wide policy is applied. The second case is explicit deny. Cilium `ingressDeny` and `egressDeny` rules set the affected source–destination cells to deny rather than allow but otherwise follow the same ingress and egress processing logic. Because these rules have the highest priority and can override previously applied policies, they are applied in the final phase and take precedence during later query evaluation.

5.3 Connectivity Query

After the reachability matrix is built, the Reachability Creator answers connectivity questions of the form “can source *S* reach destination *D*?”. A query may specify source and destination namespaces, individual workloads, or workloads together with a destination port and protocol. This step is needed because the matrix stores only the sources and targets affected by the given network policies. Many verification questions therefore involve endpoints that are not represented as a single explicit matrix cell but must be resolved through the wildcard endpoints created during matrix construction.

Query Resolution The logic behind connectivity queries can be summarized as follows: first identify all matrix rows and columns that match the query, then determine whether the flow is allowed based on the policy layer that governs those cells. A single query may match several cells at different levels of specificity, so the result is not taken from the first matching cell. Instead, the Reachability Creator evaluates all applicable matches and returns the outcome from the highest-priority policy layer.

Endpoint Matching The first stage matches the query against matrix rows and columns. For the source, a namespace-level query matches `namespace_*_**` row when the namespace equals the query namespace. If a source workload is also specified, workload-level rows such as `namespace_workload_**` are included when the workload identity matches. For cluster-wide policies, rows with a wildcard namespace may also match. Source rows never encode port or protocol because network policies do not constrain origin ports. For the destination, the same namespace and workload logic applies but a port and protocol may also be specified. In addition to namespace-level and workload-level columns, the Reachability Creator considers specific endpoints such as `namespace_workload_port_protocol`, as well as broader wildcard forms created during aggregation, including `namespace_workload_port_*` and `namespace_workload_**`. Port and protocol matching supports exact values and wildcards, so a query for TCP port 3001 can match `namespace_workload_3001_TCP`, `namespace_workload_3001_*`, `namespace_workload_**` and `namespace_*`.

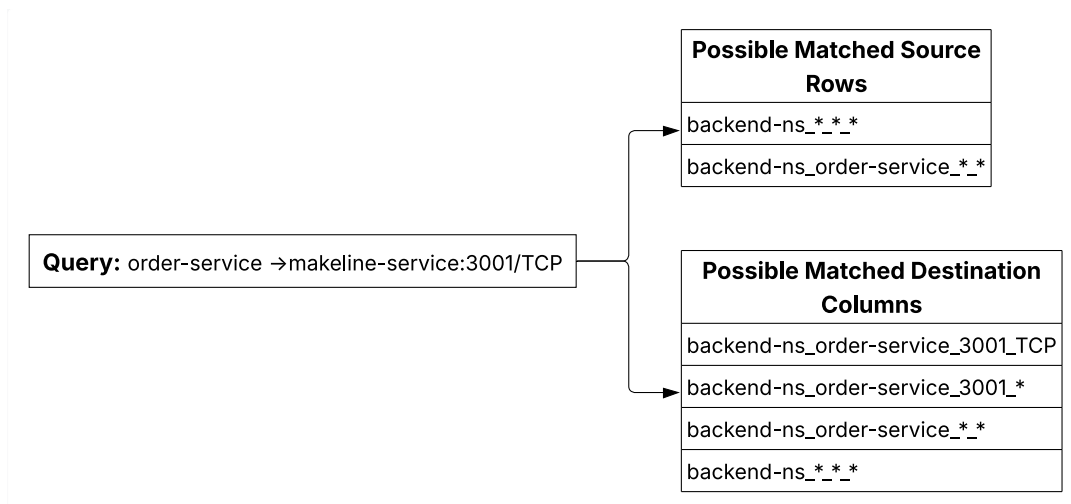


Figure 22: Example of all matrix endpoints matched by a connectivity query

_ endpoints. Figure 22 shows an example query together with all source rows and destination columns it matches. If no matching source row or destination column exists, the query is invalid. Otherwise, all matching source–destination pairs are evaluated in the matrix.

Priority Resolution The second stage resolves the query result from the matching cells. At query time, each matching cell is mapped to one of four priority tiers from highest to lowest: Cilium deny, policy allow, policy deny, and default connectivity. These tiers are not encoded in the reachability matrix itself, which stores only allow/deny values (1 or 0); instead, they are derived from the cell value together with auxiliary metadata maintained alongside the matrix, recording which endpoints were affected by network policies and which source–destination pairs were denied by Cilium explicit deny rules. A cell covered by a Cilium deny record is classified as Cilium deny; a cell whose endpoint was marked as policy-controlled is classified as policy allow or policy deny according to its matrix value; and remaining cells fall into the default tier. This ordering reflects the behaviour described in Sections 2.2 and 2.4: Cilium explicit deny rules override other decisions; policy-controlled allow and deny cells follow; and namespace-level default connectivity has the lowest priority. For deny tiers, if any matching cell in that tier is denied, the query is not reachable. For allow tiers, if any matching cell in that tier is allowed, the query is reachable. If no higher-priority tier decides the outcome, the default tier is used. The final answer therefore reflects the highest-priority tier that applies to the query.

Representative Query Scenarios Figure 23 shows selected connectivity queries from the project’s evaluation suite. Each example is representative of a distinct resolution behaviour of `query_reachability()`: wildcard endpoint matching, selection among multiple applicable matrix cells, or priority-tier resolution across `cilium_deny`, `policy_allow`, `policy_deny`, and `default`. The following paragraphs explain how these scenarios are resolved in practice.

- **Workload-to-workload query:** Consider an egress policy that allows `order-service` in `backend-ns` to reach `makeline-service` only on TCP port 3001. A query from `order-service` to `makeline-service` on TCP port 3001 is reachable because the matching allow cell is found at the policy-allow tier. A query from the same source to another workload such as `product-service`, without specifying a port, is not reachable: the query matches the wildcard destination column for that workload but the restricted egress row denies all destinations that are not explicitly allowed.
- **Port-specific query:** When a policy permits traffic to a destination workload only on specific ports, the query must match that port and protocol even if the destination workload is correct. For example, a namespace-selector egress policy may allow workloads in `backend-ns` to reach `rabbitmq` in `database-ns` on TCP port 5672. A query from `order-service` to `rabbitmq` on port 5672 is reachable, whereas the same query on port 27017 is not reachable.

Scenario	Policy	Query	Result
Workload-to-workload	Egress: order-service → makeline-service TCP/3001	order-service → makeline-service:3001	Reachable (policy allow)
Workload-to-workload	Same	order-service → product-service	Not reachable (policy deny)
Port-specific	Namespace egress: backend-ns → rabbitmq TCP/5672	order-service → rabbitmq:5672	Reachable (policy allow)
Port-specific	Same	order-service → rabbitmq:27017	Not reachable (policy deny)
Cross-namespace	Namespace ingress: backend-ns	order-service → mongodb:27017	Reachable (policy allow)
Cross-namespace	Same	store-front → mongodb:27017	Not reachable (policy deny)
Unspecified port	Egress: order-service → makeline-service TCP/3001	order-service → product-service (no port)	Not reachable (policy deny)
Cilium explicit deny	Allow: reviews-v1 → ratings-v1 TCP/9080 Deny: productpage-v1 → ratings-v1 TCP/9080	reviews-v1 → ratings-v1:9080	Reachable (policy allow)
Cilium explicit deny	Same	productpage-v1 → ratings-v1:9080	Not reachable (Cilium deny)

Figure 23: Representative connectivity queries and their resolution outcomes

- **Cross-namespace query:** Cross-namespace connectivity is resolved using the same endpoint-matching and priority rules. For example, a namespace-selector ingress policy may allow workloads in `backend-ns` to reach `mongodb` in `database-ns` on TCP port 27017. A query from `order-service` in `backend-ns` to that destination on the allowed port is reachable. A query from `store-front` in `frontend-ns` to the same destination is not reachable because the source namespace is outside the permitted selector.
- **Query without a specified port:** If a query specifies source and destination workloads but omits the port and protocol, the Reachability Creator matches broader destination columns, including wildcard forms such as `namespace_workload_.*`. The result then depends on whether any matching cell at the applicable priority tier is allowed. For example, when `order-service` may egress only to `makeline-service` on TCP port 3001, a query from `order-service` to `product-service` without a port is not reachable because only the wildcard destination column matches and that cell is denied.
- **Cilium explicit deny:** When resolving a query, the Cilium-deny tier has highest priority. For example, a Cilium ingress policy on `ratings-v1` may allow `reviews-v1` on TCP port 9080 while an `ingressDeny` rule blocks `productpage-v1` to the same destination on the same port. The first query is resolved at the policy-allow tier; the second is resolved at the Cilium-deny tier, even though both queries target the same destination workload and port.

5.4 Policy Simplification

The goal of the Policy Simplifier is to determine whether two network policy configurations induce the same connectivity for the same application topology. This supports two main use cases: detecting redundant or superseding policies by comparing cluster state with and without a given policy or rule, and identifying functionally equivalent network policies across Kubernetes and Cilium representations.

The Policy Simplifier takes three inputs: an application YAML directory, a first network policy YAML file or directory, and a second network policy YAML file or directory. For each policy set, it builds a reachability matrix using the Reachability Creator and then compares the two results. The component reports whether the two policy sets are equivalent and, if not, which source–destination pairs differ.

Comparison Strategy The comparison must account for the fact that two syntactically different policy sets may introduce different explicit matrix endpoints while still producing identical connectivity. For this reason, the Policy Simplifier uses a hybrid strategy rather than comparing only raw matrix labels.

When a source and destination appear in both reachability matrices, their stored matrix values are compared directly. When at least one endpoint appears in only one matrix, for example because port-specific rules introduced additional rows or

Scenario	Policy Set A	Policy Set B	Equivalent?
K8s vs Cilium	K8s ingress: details TCP/9080	Cilium ingress: details → TCP/9080	Yes
Non-equivalent	order-service → makeline-service TCP/3001	order-service → makeline-service (all ports)	No
Redundant rule	order-service → makeline-service (all ports)	Policy Set A + order-service → makeline-service TCP/3001	Yes

Figure 24: Policy comparison scenarios

columns, connectivity is evaluated semantically through the Reachability Creator’s query component on both policy sets. This ensures that differences are detected at the level of actual reachability, not merely at the level of matrix structure.

Comparison Process The Policy Simplifier first identifies the rows and columns shared by both matrices and the endpoints that are exclusive to one side. It then evaluates every source–destination pair in the union of both endpoint sets. Pairs whose source and destination both belong to the shared endpoint space are compared using the matrix values directly. All other pairs are compared by resolving connectivity queries on both policy sets and comparing the resulting reachability outcomes. When a pair differs, it is recorded together with the value from each side and the comparison method used.

Representative Comparison Scenarios Table 24 summarizes three representative comparisons. Each row shows two policy sets evaluated against the same application topology, together with whether the Policy Simplifier reports them as equivalent.

The following paragraphs explain each row in more detail.

- **Equivalent Kubernetes and Cilium policies:** As shown in the first row of Table 24, two policy sets may express the same restriction in different policy languages. A Kubernetes ingress policy that permits inbound traffic to `details` pods only on TCP port 9080 and an equivalent Cilium ingress policy with the same endpoint selector and port restriction should produce the same connectivity. The simplifier therefore reports them as equivalent even though the YAML syntax differs.
- **Non-equivalent policies:** The second row of Table 24 shows two policy sets that restrict the same source and destination differently. If policy set A allows `order-service` to reach `makeline-service` only on TCP port 3001, while policy set B allows the same source to reach the same destination on all ports, the two configurations are not equivalent. At least one source–destination pair has a different reachability outcome, so the simplifier reports a mismatch.
- **Redundant or superseding rules:** The third row shows the opposite situation. If policy set A already allows `order-service` to reach `makeline-service`

on all ports, adding a second egress rule that allows the same flow only on TCP port 3001 does not introduce any new connectivity. The port-specific rule is redundant, and the simplifier reports the two configurations as equivalent.

6 Evaluation

This chapter evaluates the tool in two parts. Section 6.1 assesses whether the implementation is semantically correct, and Section 6.2 assesses whether it remains practical as the number of policies grows. The evaluation is tied directly to the three contributions introduced in Section 4: (1) reachability matrix construction, (2) connectivity queries over that matrix, and (3) policy simplification through reachability comparison.

Unlike benchmark suites that reuse an existing public oracle, this work does not rely on a pre-existing database of annotated network-policy cases. Instead, the test scenarios were designed and created for this thesis. As described in Section 4, we selected two reference applications and defined policy scenarios that exercise the policy constructs supported by the tool. For each scenario, the expected outcome was derived manually from the policy semantics and application topology, then stored as a machine-readable oracle. The goal of the evaluation is therefore not to report how many files were tested but to show that each contribution behaves as intended on representative, manually validated cases.

All correctness tests are *static*: they load application YAML manifests and policy YAML files from disk, run the tool locally, and compare the output with the stored oracle. No Kubernetes cluster is required at test time. The application manifests are always applied in the sense that the Workload Discoverer parses the full application topology before any policy is evaluated; correctness therefore depends on both policy interpretation and topology parsing.

The complete test suite, including application manifests, policy files, oracles, and test scripts, is available in our [GitHub repository](#).

6.1 Correctness

6.1.1 Test Design and Validation Approach

We evaluate correctness on the two reference applications used throughout this thesis: *AKS Store Demo* and *Istio Bookinfo*. Each application is combined with a set of policy scenarios and expected-result oracles to form a *testbed*.

The test scenarios were created manually in three steps. First, we identified the policy constructs that the tool claims to support, based on the Kubernetes and Cilium policy models presented earlier in the thesis. These include pod selectors, namespace selectors, ipBlock/CIDR rules, port restrictions, port ranges, ingress and egress direction, combined ingress–egress policies, and Cilium-specific constructs such as explicit deny, cluster-wide scope, and CIDR-based ingress/egress rules. Second, for each construct we wrote one or more small policy YAML files that isolate that behaviour on a fixed application topology. Third, we manually determined the expected connectivity outcome for each scenario and encoded it as an oracle. For reachability matrix tests, the oracle is a Comma-Separated Values (CSV) file representing the full expected matrix. For connectivity query tests, the oracle is a JavaScript Object Notation (JSON) file listing individual queries together with the expected reachable

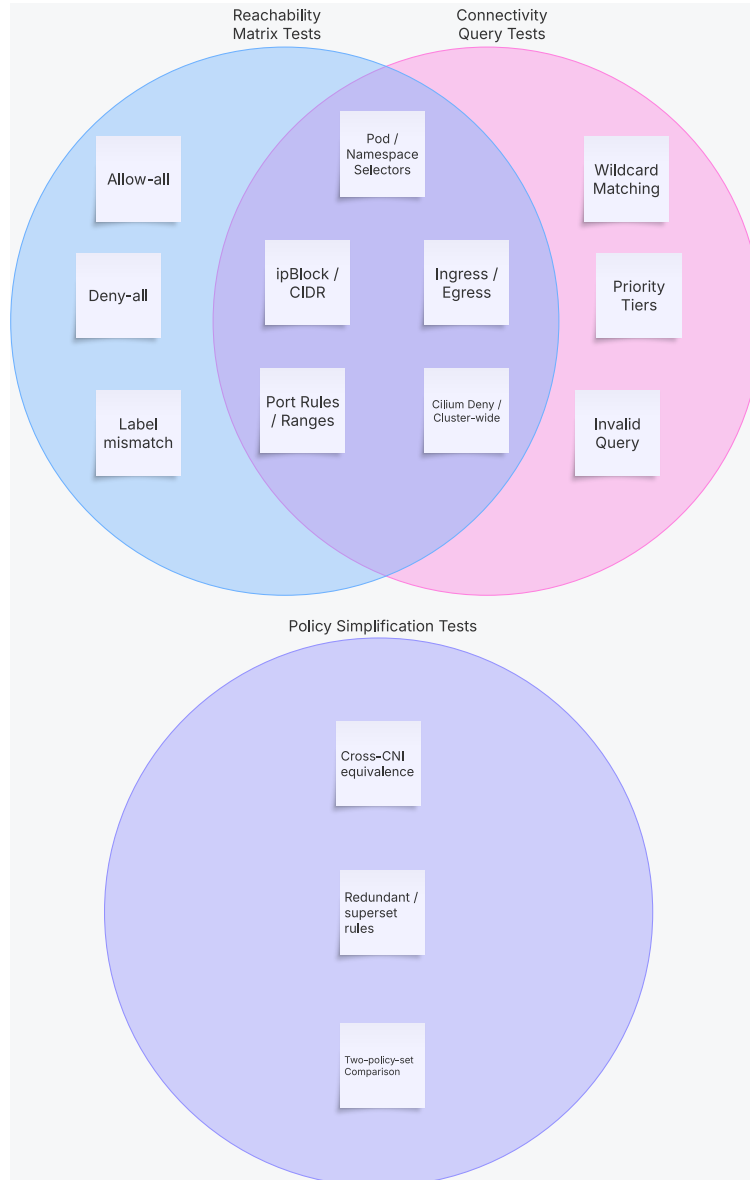


Figure 25: Coverage of the correctness evaluation by test category

outcome and deciding tier. For policy simplification tests, the oracle is a JSON file stating whether two policy sets should be equivalent.

This manual annotation process is the source of ground truth in the evaluation. The automated test does not re-derive the expected result; it only checks whether the tool output matches the manually annotated oracle. A passing test therefore means that the implementation agrees with the intended policy semantics on that scenario. Where namespace-selector behaviour required workloads in multiple namespaces, we modified the namespace assignments of selected components relative to the default application YAML.

Figure 25 summarizes how the correctness evaluation is organized by policy category and evaluation target. The three circles represent reachability matrix tests,

connectivity query tests, and policy simplification tests. Reachability matrix tests validate the full matrix output against CSV oracles and additionally cover matrix-only categories that are not exercised by the query suite: allow-all and deny-all baselines, as well as label-mismatch scenarios. Connectivity query tests validate query resolution on an already built matrix and cover query-only categories: wildcard endpoint matching, priority tier resolution, and invalid query handling. Policy simplification tests validate equivalence detection between two policy sets on the same topology and cover simplifier-only categories: cross-CNI equivalence, redundant or superset rules, and two-policy-set comparison. The overlap between reachability matrix and connectivity query tests covers shared policy categories, including pod and namespace selectors, ipBlock/CIDR rules, ingress and egress direction, port rules and ranges, and Cilium deny and cluster-wide policies. Many scenarios therefore reuse the same policy file across matrix and query tests, so the evaluation targets overlap in the policy constructs they cover while still testing different tool functions.

Test execution workflow Validation is fully automated but static. For each test case, the test script loads the application manifests for the selected testbed, parses the policy input, runs the relevant tool function, and compares the result with the oracle. Matrix tests require an exact match between the produced matrix and the CSV oracle. Query tests compare both the `reachable` flag and the deciding `tier`. Simplification tests compare the `equal` flag with the JSON oracle. If any case fails, the script reports the mismatch and the overall run fails. The correctness runner is `testing/scenario_test.py`.

Evaluation objectives The three correctness targets answer three different questions:

1. **Reachability matrix validation** asks whether the tool builds the correct connectivity representation after policy application. This is the foundation of the first contribution: if the matrix is wrong, both queries and simplification will also be wrong.
2. **Connectivity query validation** asks whether the tool answers individual reachability questions correctly once the matrix has been built. This validates the second contribution, including endpoint matching, wildcard resolution, and priority tiers.
3. **Policy simplification validation** asks whether the tool correctly decides whether two policy sets induce the same connectivity. This validates the third contribution, including the hybrid matrix-and-query comparison strategy.

6.1.2 Reachability Matrix Validation

The purpose of the reachability matrix tests is to verify that the Reachability Creator produces the same connectivity representation as the manually annotated ground truth for each isolated policy scenario. For each case, we manually traced how the policy

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: pod-selector-egress-ingress
  namespace: backend-ns
spec:
  podSelector:
    matchLabels:
      app: makeline-service
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: order-service
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: product-service

```

Figure 26: Example policy scenario for reachability matrix tests

```

, database-ns_*, backend-ns_*, frontend-ns_*, backend-ns_makeline-service_*, backend-ns_product-service_*
database-ns_*, 1, 1, 1, 0, 0, 1, 1
backend-ns_*, 1, 1, 1, 0, 0, 1, 1
frontend-ns_*, 1, 1, 1, 0, 0, 1, 1
backend-ns_order-service_*, 1, 1, 1, 1, 1, 1, 1
backend-ns_makeline-service_*, 0, 0, 0, 0, 0, 1, 1

```

Figure 27: CSV oracle for the example

should affect the fixed application topology and recorded the resulting allowed and denied source–destination relationships in a CSV oracle. The automated test then builds the matrix with the tool and checks for an exact match. A failure indicates a semantic disagreement between the implementation and the intended policy effect.

The matrix suite is organised by policy engine and selector type. The Kubernetes group covers podSelector, namespaceSelector, and ipBlock scenarios, including ingress and egress rules, with and without ports, port ranges, label mismatches, and combined ingress–egress policies. The Cilium group covers equivalent endpoint-selector scenarios plus Cilium-specific behaviour such as ingress deny, egress deny, allow/deny precedence, cluster-wide policies, and CIDR-based rules. Baseline allow-all and deny-all cases are included for both engines.

Figures 26 and 27 present a representative NetworkPolicy and its corresponding ground-truth reachability matrix. The example is representative because it captures the common evaluation pattern in our suite: a namespace-scoped policy that targets a specific microservice via a pod label selector (here, makeline-service in backend-ns) and defines both ingress and egress constraints using label-based peer selection (ingress allowed only from order-service, egress allowed only to product-service). Fig. 27 then encodes the resulting allowed/denied flows as a binary matrix. All reachability-matrix cases in the suite pass successfully. All reachability matrix cases in the suite pass

```

1 {
2   "policy_file": "pod-selector-ingress-with-ports.yaml",
3   "queries": [
4     {
5       "name": "allowed_ingress",
6       "description": "order-service may send ingress traffic to makeline-service on TCP/3001",
7       "query": {
8         "source_namespace": "backend-ns",
9         "dest_namespace": "backend-ns",
10        "source_workload": "order-service",
11        "dest_workload": "makeline-service",
12        "port": 3001,
13        "protocol": "TCP"
14      },
15      "expected": {
16        "reachable": true,
17        "tier": "policy_allow"
18      }
19    },
20    {
21      "name": "denied_ingress",
22      "description": "product-service may not send ingress traffic to makeline-service on TCP/3001",
23      "query": {
24        "source_namespace": "backend-ns",
25        "dest_namespace": "backend-ns",
26        "source_workload": "product-service",
27        "dest_workload": "makeline-service",
28        "port": 3001,
29        "protocol": "TCP"
30      },
31      "expected": {
32        "reachable": false,
33        "tier": "policy_deny"
34      }
35    }
36  ]
37 }

```

Figure 28: JSON oracle for connectivity query validation

successfully.

6.1.3 Connectivity Query Validation

Matrix validation alone does not prove that individual connectivity questions are answered correctly because queries may match multiple wildcard endpoints and must be resolved by priority tier. The connectivity query tests therefore validate the second contribution directly.

Each query case reuses a policy from the matrix suite and adds a JSON oracle that specifies one or more queries together with the expected `reachable` outcome and deciding tier. An example oracle is shown in Figure 28.

The query suite covers allowed and denied cases for pod-selector, namespace-selector, and ipBlock/CIDR policies, combined ingress–egress policies, and Cilium-specific allow, deny, cluster-wide, and CIDR-based policies. All connectivity query cases pass successfully.

6.1.4 Policy Simplification Validation

Policy simplification tests validate the third contribution: deciding whether two policy sets are equivalent on the same topology. Each comparison case provides two policy inputs and a JSON oracle with the expected `equal` result.

We include three scenario types per testbed. In *Kubernetes–Cilium equivalence*, a Kubernetes policy and an equivalent Cilium policy express the same restriction

```

1 apiVersion: networking.k8s.io/v1
2 kind: NetworkPolicy
3 metadata:
4   name: order-to-makeline-3001
5   namespace: backend-ns
6 spec:
7   podSelector:
8     matchLabels:
9       app: order-service
10  policyTypes:
11  - Egress
12  egress:
13  - to:
14    - podSelector:
15      matchLabels:
16        app: makeline-service
17  ports:
18  - protocol: TCP
19    port: 3001
20
1 apiVersion: networking.k8s.io/v1
2 kind: NetworkPolicy
3 metadata:
4   name: order-to-makeline-all-ports
5   namespace: backend-ns
6 spec:
7   podSelector:
8     matchLabels:
9       app: order-service
10  policyTypes:
11  - Egress
12  egress:
13  - to:
14    - podSelector:
15      matchLabels:
16        app: makeline-service
17
1 {
2   "equal": true,
3   "description": "Port-specific egress rule is redundant when order-service may already reach makeline-service on all ports."
4 }

```

Figure 29: Example comparison case for redundant policy detection

in different syntax; the oracle expects equivalence. In *non-equivalence*, two policy sets restrict the same source–destination flow differently—for example, allowing communication only on a specific port in one set and on all ports in the other; the oracle expects a mismatch. In *superset redundancy*, one policy set contains a broader rule and the other adds a narrower rule on top of it; the oracle expects equivalence because the additional rule is redundant (Figure 29).

All policy simplification cases pass successfully.

6.2 Performance

The purpose of the performance evaluation is complementary to correctness. While the manually designed correctness suite shows that the tool is semantically right on representative cases, the performance experiments study whether the tool remains usable in larger deployments with many network policies. Large clusters can contain hundreds or thousands of policies, so runtime and memory footprint matter even when correctness has already been established.

We therefore measure three aspects:

1. **Matrix construction time** because building the reachability matrix is the most expensive step and is required before queries or simplification can be performed.
2. **Matrix size** because the tool stores a sparse but potentially growing set of source rows and destination columns; memory use and later comparison cost depend on how many endpoints are created as policies accumulate.
3. **Query and simplification time** because these operations must remain fast enough for interactive verification once the matrix exists.

The correctness suite already contains realistic policy constructs but it is not designed to stress-test scale. For performance, we therefore use *synthetic policy loads* of 10, 25, 50, 100, and 1000 policies. These values represent a small configuration,

a medium-sized deployment, and a large policy-heavy environment, respectively. The application topology remains fixed in each testbed; only the number of policies increases. This isolates the effect of policy count on runtime and matrix growth.

The synthetic policies are generated deterministically from the fixed workload list in each testbed. For policy index i , the generator selects a source workload and a peer workload, alternates between ingress and egress rules, and adds a TCP port restriction to every fourth policy. The policies are not random: they systematically increase rule count while periodically introducing new port-specific endpoints.

We analyze performance in two complementary experiments. **Experiment 1** uses `aks-store-demo` to study runtime growth together with matrix growth. **Experiment 2** uses `istio-bookinfo` to study runtime growth when the matrix size saturates because the smaller topology exposes fewer distinct endpoint identities.

6.2.1 Computational Complexity

Let P denote the number of network policy objects, R the total number of parsed rules across all policies, and E the number of endpoints in a reachability matrix.

Reachability matrix creation. Applying one rule requires mapping selectors to matrix rows and columns and updating the ingress or egress matrix. In the **best case**, a rule touches only a small number of endpoints, giving $O(E)$ per rule and $O(R \cdot E)$ overall, with space $O(E^2)$. In the **average case**, rules regularly create or update several endpoints and their wildcard parents, which remains $O(R \cdot E)$ but with a larger constant factor. In the **worst case**, a rule triggers updates across full rows or columns, for example during implicit deny propagation, giving $O(R \cdot E^2)$ time.

Connectivity queries. A query first matches source rows and destination columns, then resolves the result by priority tier. In the **best case**, a workload-specific query matches only a few endpoints, giving $O(1)$ to $O(E)$ time. In the **average case**, several wildcard endpoints match and must be inspected, giving $O(E)$. In the **worst case**, a broad namespace-level query matches many row-column pairs, giving $O(E^2)$ time. Extra space beyond the stored matrix is $O(1)$.

Policy simplification. The simplifier builds two reachability matrices and compares the union of their endpoint sets. In the **best case**, most pairs belong to shared endpoints and are resolved by direct matrix lookup after two matrix builds, giving cost dominated by $O(R_a \cdot E + R_b \cdot E)$. In the **average case**, some exclusive endpoints require query-based comparison on both sides, adding cost proportional to the number of union pairs resolved by query. In the **worst case**, matrix construction dominates with $O(R_a \cdot E^2 + R_b \cdot E^2)$, and query-based comparisons add further cost on exclusive endpoint pairs.

6.2.2 Performance Benchmark Setup

For each testbed, the benchmark repeats three measurements at every policy load:

Policies	Time (ms)	Rows	Cols	Cells
10	17.54	7	15	105
25	20.94	7	23	161
50	48.35	7	35	245
100	221.51	7	59	413
1000	6531.24	7	109	763

Table 4: Average matrix creation time and size performance for aks-store-demo

Policies	Time (ms)	Rows	Cols	Cells
10	9.22	5	17	85
25	12.97	5	25	125
50	37.74	5	37	185
100	118.80	5	61	305
1000	1827.86	5	61	305

Table 5: Average matrix creation time and size for istio-bookinfo

1. **Matrix creation.** The reachability matrix is built **three times** per testbed; we report the **mean wall-clock time over 3 runs** together with the matrix dimensions.
2. **Query lookup.** After each matrix build, we execute **1000** connectivity queries and report the **mean lookup time per query, averaged over the 3 runs**. Queries rotate across five predefined query types per testbed.
3. **Policy simplification.** Two synthetic policy sets of equal size are compared. The benchmark reports total time, comparison-only time, and the number of source–destination pairs in the union of both matrices. Each measurement is averaged over three runs.

6.2.3 Performance Results

We evaluate both testbeds at 10, 25, 50, 100, and 1000 synthetic policies. All reported times are means over three runs.

Matrix creation. Tables 4 and 5 report mean matrix-creation time together with the resulting matrix dimensions.

On aks-store-demo, matrix rows remain fixed at seven because the workload topology does not change. Columns grow because the synthetic generator introduces new port-specific endpoints as policy count increases. Both matrix size and construction time therefore scale with policy count.

On istio-bookinfo, the pattern is different. Between 10 and 100 policies, matrix size grows from 5×17 to 5×61 cells. Beyond 100 policies, the matrix saturates

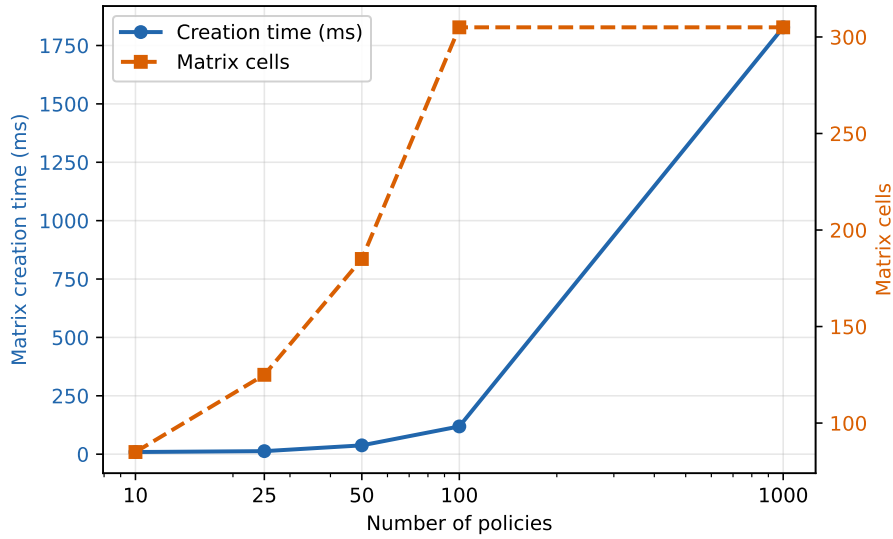


Figure 30: Matrix creation time and size for *istio-bookinfo*

Policies	Average lookup time (ms)
10	0.11
25	0.10
50	0.13
100	0.27
1000	0.50

Table 6: Average query lookup time for *aks-store-demo*

at 5×61 , yet creation time continues to rise from 118.80 ms to 1827.86 ms. This separates two cost components: *topology expansion* (new endpoint identities) and *rule processing* (parsing and applying each policy even when no new endpoint is introduced). The intermediate points at 25 and 50 policies make this transition visible before saturation. Figure 30 plots both effects on *istio-bookinfo*.

Query lookup. Tables 6 and 7 report mean lookup time per query after each matrix build. Each load executes 1000 queries, rotating across five predefined query types; the reported value is the mean per query, averaged over three runs.

Query lookup remains fast on both testbeds. Average time grows modestly with policy count and stays below one millisecond per query in all reported runs. On *istio-bookinfo*, lookup time stabilizes once the matrix saturates, whereas on *aks-store-demo* it continues to grow with matrix size.

Policy simplification. Table 8 reports simplification cost for both testbeds. For each policy load, two synthetic policy sets of equal size are compared. Total time includes building both reachability matrices; compare time measures only the pairwise

Policies	Average lookup time (ms)
10	0.12
25	0.10
50	0.13
100	0.25
1000	0.28

Table 7: Average query lookup time for `istio-bookinfo`

Testbed	Policies	Total (ms)	Compare (ms)	Union pairs
aks-store-demo	10	53.48	5.41	105
aks-store-demo	25	63.06	5.54	161
aks-store-demo	50	114.72	7.81	245
aks-store-demo	100	378.27	23.22	427
aks-store-demo	1000	10653.62	16.14	763
istio-bookinfo	10	32.08	7.10	85
istio-bookinfo	25	39.17	7.57	125
istio-bookinfo	50	68.18	10.77	185
istio-bookinfo	100	217.80	6.51	305
istio-bookinfo	1000	3060.26	6.58	305

Table 8: Average policy simplification time for both testbeds

comparison step.

Policy simplification is dominated by building the two reachability matrices. Total time tracks matrix-creation cost and grows with both policy count and union-pair count. The comparison step itself remains inexpensive because most endpoint pairs are resolved by direct matrix lookup on shared rows and columns. On `istio-bookinfo`, compare time stays near 6–11 ms even at 1000 policies, when matrix size has already saturated but matrix construction has not.

7 Discussion

This chapter discusses the practical implications of the tool, the main challenges encountered during implementation, the limitations of the current design, and possible directions for future work.

7.1 Practical Implications

The main achievement of this work is a practical offline verifier for Kubernetes network-policy reachability. Rather than treating policy analysis as a formal exercise alone, the tool is designed to answer concrete questions that operators and security engineers face in real environments: whether one workload can reach another, under which port and protocol, and how that changes when policies are added, removed, or rewritten.

A first practical contribution is *pre-deployment verification*. Because the tool works entirely from application manifests and policy YAML, it can be used during design or review before changes reach a live cluster. This makes it possible to detect unintended exposure or blocked service paths early, when correction is still inexpensive. The reachability matrix therefore functions not only as a verification artifact but as an explicit model of effective connectivity across the application topology.

A second contribution is *interactive policy analysis*. Connectivity queries make it practical to ask targeted what-if questions during policy design, such as whether a backend service can reach a database on a specific port, or whether a new frontend rule opens cross-namespace access. This shifts network-policy reasoning from runtime debugging to a design-time activity that can be repeated as policies evolve.

A third contribution is *policy auditing and simplification*. The policy simplifier addresses a common operational problem: two policy sets may look different in YAML while inducing the same connectivity, or one set may contain redundant rules that add complexity without changing behaviour. By comparing policy sets semantically rather than syntactically, the tool can support cleanup, migration, and review workflows in clusters where policies accumulate over time.

A fourth contribution is *CNI-aware offline analysis*. Many production clusters rely on Cilium features such as cluster-wide scope and explicit deny rules, which extend the native Kubernetes allow-only model. Supporting these constructs makes the tool relevant to realistic environments, where effective connectivity is determined by layered allow and deny semantics rather than by Kubernetes defaults alone.

Overall, the practical implication is that network-policy reasoning can be made explicit, repeatable, and usable before deployment. The same reachability representation supports verification, investigation, and refinement, turning policy analysis from an ad hoc cluster activity into a systematic part of secure application design. These benefits should be interpreted in light of the assumptions and scope constraints discussed in the following subsections.

7.2 Challenges

The first challenge was how to represent reachability in a form suitable for Kubernetes verification. Prior work, as reviewed in Section 3, uses many different notions of source and destination (routers, IP addresses, ports, workloads) and different structures such as matrices or graphs. We aligned our model with Kubernetes Network Policies, which select pods by labels and can restrict traffic by port and protocol. Accordingly, endpoints are encoded as namespace, workload, port, and protocol. We maintain separate ingress and egress matrices to reflect how Kubernetes evaluates traffic direction, and combine them using matrix multiplication to obtain the final reachability result. To keep the matrix manageable, we include only endpoints referenced in the parsed policies, rather than every possible source–destination pair in the cluster.

Another challenge was supporting ipBlock rules. Because the tool performs offline control-plane analysis, it cannot know which IP addresses are assigned to pods at runtime, and therefore cannot determine in advance whether a given CIDR range matches any live pod address. We address this by representing external IP ranges directly as endpoints in the reachability matrix, using the CIDR itself in the endpoint encoding. Connectivity to a specific IP can then be evaluated at query time by checking whether that address falls within a stored CIDR endpoint.

Another challenge was determining whether two reachability matrices represent the same connectivity semantics in the policy simplifier. Because of how the tool builds matrices incrementally, two semantically equivalent policy sets can still produce structurally different matrices. For example, in Figure 29, the first case includes both network policies, while the second includes only the second policy. The resulting matrices differ because the first case also adds more specific endpoints with explicit ports and protocols, together with their parent wildcard endpoints to the matrix, even though connectivity queries should return the same results. To address this, the policy simplifier uses a hybrid comparison strategy. For source–destination pairs whose endpoints appear in both matrices, it compares the stored matrix values directly. For pairs involving rows or columns that exist in only one matrix, it falls back to the `connectivity query` functionality of `ReachabilityCreator` and compares the query results on both policy sets. This makes it possible to detect semantic equivalence even when the matrices themselves are not identical.

The final challenge was supporting Cilium cluster-wide and deny policies. Standard Kubernetes NetworkPolicies are namespace-scoped and, in practice, only express allow rules in an additive model. Cilium extends this through its own CRDs, including `CiliumClusterwideNetworkPolicy`, which can affect matching workloads and destinations across namespaces, and explicit deny rules (`ingressDeny` and `egressDeny`), which contradict the allow-based semantics of native Kubernetes policies. To handle this, our tool applies policies in phases so that Cilium deny rules are evaluated after allow rules and can override them. During connectivity queries, Cilium deny decisions are also given the highest priority. For cluster-wide policies, namespace wildcards are used in endpoint encoding, allowing a single rule to represent matches that span multiple namespaces.

7.3 Limitations

Although the preceding section described how several implementation challenges were addressed, the resulting tool still operates within clear boundaries. The limitations in our work are related to both the information conveyed in application manifests and network policies, and to the expressivity of the policy formats the tool supports. In other words, some conclusions cannot be drawn because the static inputs do not fully describe runtime behaviour [4][5], while others cannot be drawn because certain policy languages lie outside the current scope.

The first limitation is that manifests and policies do not necessarily reveal all open ports in the application topology. Our tool assumes that the analyzed application topology and network policies capture all relevant open ports. In practice, Kubernetes configuration files are not binding: a workload may listen on ports that are not declared in its Deployment or Service manifests. Without this assumption, however, it would not be possible to draw meaningful conclusions about application reachability because the set of exposed endpoints would be unknown.

A second limitation concerns `ipBlock` rules. Because control-plane analysis is performed offline, pod IP addresses are not available at analysis time. As a result, the tool cannot determine whether a CIDR specified in an `ipBlock` rule matches a pod address or not. Although CIDR ranges can be represented in the reachability matrix and queried later for specific IP addresses, the real runtime effect of an `ipBlock` rule on pod-to-pod connectivity cannot be resolved fully offline.

A third limitation concerns expressivity: the tool supports standard Kubernetes `NetworkPolicy` resources and Cilium CRDs but not other popular CNI implementations such as Calico. Policies expressed in CNI-specific formats outside this scope are therefore not analyzed.

Finally, the evaluation itself was conducted in a constrained environment. All automated correctness and performance experiments were executed locally on an Apple M2 laptop with 16 GB RAM, running macOS 26.0.1. This setup was chosen for reproducibility and ease of iteration during development but the reported runtimes therefore reflect offline analysis on a single developer workstation rather than execution within a production cluster-management pipeline or on heterogeneous infrastructure. As a result, the performance results should be interpreted as indicative scalability trends under increasing policy load, rather than as definitive bounds on operational deployment cost in every environment.

7.4 Future Work

Several directions could extend the work presented in this thesis, spanning both the reachability analysis pipeline and the policy comparison capabilities developed in the tool. Building on this foundation, future work could broaden policy support, deepen the comparison workflow, and improve how connectivity is represented and evaluated at scale.

The most important next step is a stronger evaluation methodology that does not depend on a small number of manually annotated examples. The current correctness

assessment relies on hand-crafted oracles for selected policy scenarios. Scaling this approach to a large dataset would require automated ground-truth annotation, for example by deriving expected connectivity from an independent reference source rather than from manual inspection. As a first step in that direction, the offline reachability results could be compared systematically with outputs from runtime probing tools such as Kubesonde [4]. An early integration path already exists in this project, but a full comparison was not completed because of the overall time limitations of the thesis. Addressing this gap would reduce reliance on manual oracles while still validating the offline model against observed cluster behaviour. Beyond empirical evaluation, a proof of correctness in an abstract model would also be desirable, although Kubernetes and CNI semantics are complex enough that a complete formalization may not be feasible; a restricted model covering selected policy constructs could nevertheless provide useful guarantees.

Beyond evaluation, support could be broadened to additional CNIs, such as Calico, so that policies outside the Kubernetes and Cilium formats can also be analysed. The policy simplifier could also go beyond detecting redundant and equivalent policy sets and generate simplified network policies directly from a given reachability matrix. That would turn the tool from an analysis-only verifier into a policy refinement or synthesis component.

Performance results further show that the reachability matrix grows quickly as the number of network policies increases. Alternative representations, such as graphs or decision trees with different endpoint selection orders, could be explored to reduce memory usage and comparison cost while preserving the same connectivity semantics. Finally, a partial evaluation strategy could be developed for cases where connectivity cannot be resolved statically, for example when a query depends on a specific IP address or other runtime information. Such an approach would allow the tool to return conditional results rather than requiring fully concrete endpoint information upfront.

8 Conclusions

Kubernetes is now the standard platform for running containerized microservices in production, and network policies are a central security control for restricting communication between pods. As deployments grow across multiple namespaces and services, however, it becomes increasingly difficult for operators to determine what connectivity a given policy set actually permits. Policies may interact in non-obvious ways across ingress and egress directions, may be expressed in different CNI-specific formats, and may contain redundant or inconsistent rules that increase management overhead without improving security [5, 34]. Existing network verification tools developed for traditional IP networks reason over routers, switches, and forwarding tables rather than over label-selected workloads and declarative YAML manifests. Even Kubernetes-aware approaches typically depend on a live cluster, observed traffic, or a single policy format. There remains a need for offline, proactive verification that can analyze declared application and policy configurations before they are deployed.

This thesis addressed that need by designing and implementing an offline control-plane verification framework for Kubernetes network policies. The framework takes two inputs: application YAML manifests that describe cluster topology and network policy YAML files that define traffic rules, and analyses them without requiring a running cluster or live traffic. At its core, the tool normalizes native Kubernetes `NetworkPolicy` resources and Cilium policy CRDs into a common internal representation, then constructs a reachability matrix that encodes allowed and denied communication between cluster endpoints at namespace, workload, port, and protocol granularity. Separate ingress and egress matrices reflect how Kubernetes evaluates traffic direction, and their combination yields the final connectivity model. Cilium-specific semantics, including explicit deny rules and cluster-wide policies, are handled through ordered policy application phases and priority-based query resolution. On top of this model, the tool provides three capabilities: a reachability matrix for inspecting the overall communication landscape, connectivity queries for answering individual source–destination reachability questions, and a policy simplifier for comparing two policy sets to detect semantic equivalence and redundant rules, including across Kubernetes and Cilium syntax. The implementation comprises a Workload Discoverer, a Policy Parser, a Reachability Creator, a connectivity query component, and a Policy Simplifier, all implemented in Python.

We evaluated the tool on two reference microservice applications, `aks-store-demo` and `istio-bookinfo`, using manually annotated test oracles derived from policy semantics and application topology. The correctness evaluation covered pod, namespace, and `ipBlock/CIDR` selectors, ingress and egress rules, port restrictions and ranges, combined policies, and Cilium-specific constructs such as explicit deny, allow/deny precedence, and cluster-wide scope. All reachability matrix, connectivity query, and policy simplification test cases passed successfully, confirming that the implementation agrees with the intended policy semantics on representative scenarios. The performance evaluation used synthetic policy loads of 10, 25, 50 100, and 1000 policies. Matrix construction time increased with policy count, reaching approximately 6.5 seconds for 1000 policies on `aks-store-demo` and 1.8 seconds on

`istio-bookinfo`, while connectivity queries remained fast throughout, with average lookup times below one millisecond. Policy simplification was dominated by the cost of building two reachability matrices, whereas the comparison step itself remained inexpensive. Together, these results show that the proposed approach is semantically correct on the tested scenarios and remains practical for interactive verification at moderate policy scale.

The current design also has important limitations. The analysis assumes that application manifests accurately declare exposed ports and workload labels; undeclared listening ports cannot be inferred offline. Because verification is performed statically, pod IP addresses are not available at analysis time, so the full runtime effect of `ipBlock` and CIDR rules on pod-to-pod connectivity cannot be resolved without additional runtime information. Finally, the tool supports standard Kubernetes `NetworkPolicy` resources and Cilium CRDs but not other CNI policy formats such as Calico or Antrea, and it does not model layer 7 policy constraints. These boundaries define the scope within which the reported results should be interpreted.

Several directions could extend this work in future research. Most importantly, the evaluation could move beyond the current small set of manually annotated examples toward larger datasets with automated ground-truth generation, for instance by comparing offline reachability results with runtime probing tools such as Kubesonde. Support could also be broadened to additional CNIs so that a wider range of policy languages can be analyzed within the same framework. The policy simplifier could be extended from detecting equivalence and redundancy to synthesizing simplified policies directly from a reachability matrix, turning the tool from an analysis-only verifier into a policy refinement component. Alternative connectivity representations, such as graphs or decision trees, may reduce memory use and comparison cost as policy sets grow further. Finally, partial evaluation could allow the tool to return conditional results when connectivity depends on runtime information such as a specific IP address, rather than requiring fully concrete endpoint data upfront. In summary, this thesis demonstrates that offline control-plane verification provides a practical and vendor-agnostic way to reason about effective connectivity in Kubernetes, and that reachability analysis over normalized policy manifests can support proactive validation, targeted connectivity queries, and semantic comparison of heterogeneous network policy configurations before deployment.

References

- [1] Antrea Authors. *Antrea Architecture*. <https://antrea.io/docs/v2.6.2/docs/design/architecture/>. Version v2.6.2. 2026.
- [2] Ryan Beckett et al. “A General Approach to Network Configuration Verification”. In: *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. SIGCOMM ’17. Los Angeles, CA, USA: Association for Computing Machinery, 2017, pp. 155–168. DOI: [10.1145/3098822.3098834](https://doi.org/10.1145/3098822.3098834).
- [3] Dalton A. Brucker-Hahn et al. “CloudCover: Enforcement of Multi-Hop Network Connections in Microservice Deployments”. In: *2024 Annual Computer Security Applications Conference (ACSAC)*. 2024, pp. 1186–1202. DOI: [10.1109/ACSAC63791.2024.00095](https://doi.org/10.1109/ACSAC63791.2024.00095).
- [4] Jacopo Bufalino, Mario Di Francesco, and Tuomas Aura. “Analyzing Microservice Connectivity with Kubesonde”. In: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE 2023. San Francisco, CA, USA: Association for Computing Machinery, 2023, pp. 2038–2043. DOI: [10.1145/3611643.3613899](https://doi.org/10.1145/3611643.3613899).
- [5] Jacopo Bufalino et al. “Inside Job: Defending Kubernetes Clusters Against Network Misconfigurations”. In: *Proceedings of the ACM on Networking* 3.CoNEXT3 (Sept. 2025). DOI: [10.1145/3749220](https://doi.org/10.1145/3749220).
- [6] Cilium Authors. *Introduction to Cilium*. <https://docs.cilium.io/en/stable/overview/intro/>. 2026.
- [7] Cilium Authors. *Kubernetes Network Policy*. <https://docs.cilium.io/en/stable/network/kubernetes/policy/>. 2026.
- [8] Cloud Native Computing Foundation (CNCF). *CNCF Annual Survey Report 2025*. 2026. URL: https://www.cncf.io/wp-content/uploads/2026/01/CNCF_Annual_Survey_Report_final.pdf.
- [9] Cloud Native Computing Foundation (CNCF). *CNCF Annual Survey—Cloud Native 2023: The Undisputed Infrastructure of Global Technology*. 2024. URL: <https://www.cncf.io/reports/cncf-annual-survey-2023/>.
- [10] CNI Authors. *Container Network Interface (CNI) Specification and Plugins*. <https://github.com/containernetworking/cni>. 2026.
- [11] Asbat El Khairi et al. “ImmuCheck: Selective Immutability for Container Escape Detection in Containerized Microservices”. In: *Proceedings of the ACM Asia Conference on Computer and Communications Security*. ASIA CCS ’26. Bangalore, India: Association for Computing Machinery, 2026, pp. 774–787. DOI: [10.1145/3779208.3807484](https://doi.org/10.1145/3779208.3807484).
- [12] Seyed K. Fayaz et al. “Efficient network reachability analysis using a succinct control plane representation”. In: *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*. OSDI’16. Savannah, GA, USA: USENIX Association, 2016, pp. 217–232.

- [13] Wes Felter et al. “An updated performance comparison of virtual machines and Linux containers”. In: *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 2015, pp. 171–172. doi: [10.1109/ISPASS.2015.7095802](https://doi.org/10.1109/ISPASS.2015.7095802).
- [14] Flannel Authors. *Flannel: A Layer 3 Network Fabric for Kubernetes*. <https://github.com/flannel-io/flannel>. 2026.
- [15] Ari Fogel et al. “A general approach to network configuration analysis”. In: *Proceedings of the 12th USENIX Conference on Networked Systems Design and Implementation*. NSDI’15. Oakland, CA: USENIX Association, 2015, pp. 469–483.
- [16] Aaron Gember-Jacobson et al. “Fast Control Plane Analysis Using an Abstract Representation”. In: *Proceedings of the 2016 ACM SIGCOMM Conference*. SIGCOMM ’16. Florianopolis, Brazil: Association for Computing Machinery, 2016, pp. 300–313. doi: [10.1145/2934872.2934876](https://doi.org/10.1145/2934872.2934876).
- [17] Isovalent and the Cilium Authors. *Cilium: Cloud-Native Networking, Security, and Observability*. <https://cilium.io/>. 2026.
- [18] Istio Authors. *Bookinfo Application*. 2024. URL: <https://istio.io/latest/docs/examples/bookinfo/> (Accessed July 1, 2026).
- [19] Peyman Kazemian, George Varghese, and Nick McKeown. “Header space analysis: static checking for networks”. In: *Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation*. NSDI’12. San Jose, CA: USENIX Association, 2012, p. 9.
- [20] Asbat EI Khairi, Andreas Peter, and Andrea Continella. “PROCATCH: Detecting Execution-Based Anomalies in Single-Instance Microservices”. In: *2025 IEEE Conference on Communications and Network Security (CNS)*. 2025. doi: [10.1109/CNS66487.2025.11194959](https://doi.org/10.1109/CNS66487.2025.11194959).
- [21] Ahmed Khurshid et al. “Veriflow: verifying network-wide invariants in real time”. In: *ACM SIGCOMM Computer Communication Review* 42.4 (Sept. 2012), pp. 467–472. ISSN: 0146-4833. doi: [10.1145/2377677.2377766](https://doi.org/10.1145/2377677.2377766).
- [22] Bom Kim, Jinwoo Kim, and Seungsoo Lee. “Exploring Security Enhancements in Kubernetes CNI: A Deep Dive Into Network Policies”. In: *IEEE Access* 13 (2025), pp. 35322–35338. doi: [10.1109/ACCESS.2025.3543841](https://doi.org/10.1109/ACCESS.2025.3543841).
- [23] Kube-router Authors. *Kube-router Introduction*. <https://www.kube-router.io/docs/introduction/>. 2026.
- [24] Kube-router Authors. *Kube-router: A lean yet powerful turnkey networking solution for Kubernetes*. <https://www.kube-router.io/>. 2026.
- [25] Kubernetes Authors. *Cluster Architecture*. <https://kubernetes.io/docs/concepts/architecture/>. 2026.
- [26] Kubernetes Authors. *Controllers*. <https://kubernetes.io/docs/concepts/architecture/controller/>. 2024.

- [27] Kubernetes Authors. *Network Policies*. <https://kubernetes.io/docs/concepts/services-networking/network-policies/>. 2026.
- [28] Kubernetes Authors. *Object Names and IDs*. <https://kubernetes.io/docs/concepts/overview/working-with-objects/names/>. The Linux Foundation, 2026.
- [29] Kubernetes Authors. *Service*. The Linux Foundation. 2026. URL: <https://kubernetes.io/docs/concepts/services-networking/service/>.
- [30] Kubernetes Authors. *What is Kubernetes?* <https://kubernetes.io/docs/concepts/overview/>. 2026.
- [31] Kubernetes Authors. *Working with Kubernetes Objects*. <https://kubernetes.io/docs/concepts/overview/working-with-objects/>. 2024.
- [32] Kubernetes Authors. *Workloads*. <https://kubernetes.io/docs/concepts/workloads/>. 2024.
- [33] Seungsoo Lee and Jaehyun Nam. “Kunerva: Automated Network Policy Discovery Framework for Containers”. In: *IEEE Access* 11 (2023), pp. 95616–95631. DOI: [10.1109/ACCESS.2023.3310281](https://doi.org/10.1109/ACCESS.2023.3310281).
- [34] Yifan Li et al. “Kano: Efficient Cloud Native Network Policy Verification”. In: *IEEE Transactions on Network and Service Management* 20.3 (2023), pp. 3747–3764. DOI: [10.1109/TNSM.2022.3229675](https://doi.org/10.1109/TNSM.2022.3229675).
- [35] Haohui Mai et al. “Debugging the data plane with anteater”. In: *ACM SIGCOMM Computer Communication Review* 41.4 (Aug. 2011), pp. 290–301. ISSN: 0146-4833. DOI: [10.1145/2043164.2018470](https://doi.org/10.1145/2043164.2018470).
- [36] Microsoft Authors. *AKS Store Demo*. 2024. URL: <https://github.com/Azure-Samples/aks-store-demo> (Accessed July 1, 2026).
- [37] Rani Osnat. *A Brief History of Containers: From the 1970s to 2017*. Online. 2018.
- [38] Claus Pahl. “Containerization and the PaaS Cloud”. In: *IEEE Cloud Computing* 2.3 (2015), pp. 24–31. DOI: [10.1109/MCC.2015.51](https://doi.org/10.1109/MCC.2015.51).
- [39] Project Antrea Authors. *Antrea: Kubernetes Networking and Security Powered by Open vSwitch*. <https://antrea.io/>. 2026.
- [40] Mohammad Reza Saleh Sedghpour, Cristian Klein, and Johan Tordsson. “An Empirical Study of Service Mesh Traffic Management Policies for Microservices”. In: *Proceedings of the 2022 ACM/SPEC on International Conference on Performance Engineering*. ICPE ’22. Beijing, China: Association for Computing Machinery, 2022, pp. 17–27. DOI: [10.1145/3489525.3511686](https://doi.org/10.1145/3489525.3511686).
- [41] Prateek Sharma et al. “Containers and Virtual Machines at Scale: A Comparative Study”. In: *Proceedings of the 17th International Middleware Conference*. Middleware ’16. Trento, Italy: Association for Computing Machinery, 2016. DOI: [10.1145/2988336.2988337](https://doi.org/10.1145/2988336.2988337).

- [42] Junxian Shen et al. “Network-Centric Distributed Tracing with DeepFlow: Troubleshooting Your Microservices in Zero Code”. In: *Proceedings of the ACM SIGCOMM 2023 Conference*. ACM SIGCOMM ’23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 420–437. DOI: [10.1145/3603269.3604823](https://doi.org/10.1145/3603269.3604823).
- [43] Tamanna Siddiqui, Shadab Alam Siddiqui, and Najeeb Ahmad Khan. “Comprehensive Analysis of Container Technology”. In: *2019 4th International Conference on Information Systems and Computer Networks (ISCON)*. 2019, pp. 218–223. DOI: [10.1109/ISCON47742.2019.9036238](https://doi.org/10.1109/ISCON47742.2019.9036238).
- [44] The Kubernetes Authors. *Services, Load Balancing, and Networking*. The Linux Foundation. URL: <https://kubernetes.io/docs/concepts/services-networking/>.
- [45] Tigera and the Calico Open Source Community. *Project Calico: Cloud-Native Networking and Security*. <https://www.tigera.io/project-calico/>. 2026.
- [46] Tigera, Inc. *About Calico*. <https://docs.tigera.io/calico/latest/about/>. 2026.
- [47] Tigera, Inc. *Tiered Policy*. <https://docs.tigera.io/calico/latest/network-policy/policy-tiers/tiered-policy>. 2026.
- [48] Hongyi Zeng et al. “Libra: divide and conquer to verify forwarding tables in huge networks”. In: *Proceedings of the 11th USENIX Conference on Networked Systems Design and Implementation*. NSDI’14. Seattle, WA: USENIX Association, 2014, pp. 87–99.